baseline-implementation-clean

November 2, 2025

1 1. Setup and Imports

[ ]:
```
# %pip install -q openai-whisper jiwer transformers sentencepiece sacrebleu TTS

import os
import sys
import json
import subprocess
import time
import glob
import random
import shutil
from pathlib import Path
from typing import Dict, List, Optional, Tuple

# --- Standard Libraries ---
import numpy as np
import cv2  # OpenCV for video processing
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torch.nn.functional as F
from tqdm.notebook import tqdm  # Progress bars

# ================================================================================
# KAGGLE ENVIRONMENT DETECTION
# ================================================================================
IS_KAGGLE = os.path.exists('/kaggle/input')
print(f"Running on Kaggle: {IS_KAGGLE}")

if IS_KAGGLE:
    # Kaggle paths
    KAGGLE_COMPONENTS_PATH = '/kaggle/input/components/pytorch/default/1'
    KAGGLE_DATA_PATH = '/kaggle/input/voxceleb2/VoxCeleb2'

    # Add components to path
```

```
    if os.path.exists(KAGGLE_COMPONENTS_PATH):
        sys.path.insert(0, KAGGLE_COMPONENTS_PATH)
        print(f" Added Kaggle components path: {KAGGLE_COMPONENTS_PATH}")
    else:
        print(f" Warning: Kaggle components path not found:␣
 ↪{KAGGLE_COMPONENTS_PATH}")

    # Verify data exists
    if os.path.exists(KAGGLE_DATA_PATH):
        print(f" VoxCeleb2 data found: {KAGGLE_DATA_PATH}")
        # List available parts
        mp4_parts = glob.glob(f"{KAGGLE_DATA_PATH}/vox2_dev_mp4_part*")
        aac_parts = glob.glob(f"{KAGGLE_DATA_PATH}/vox2_dev_aac_part*")
        print(f"  - MP4 parts: {len(mp4_parts)}")
        print(f"  - AAC parts: {len(aac_parts)}")
    else:
        print(f" Warning: VoxCeleb2 data not found: {KAGGLE_DATA_PATH}")

# ============================================================================
# IMPORT CUSTOM COMPONENTS
# ============================================================================
try:
    from asr_component import ASRComponent, extract_audio_from_video,␣
 ↪print_transcription, export_to_srt
    from mt_component import MTComponent, print_translation_summary,␣
 ↪export_translation_to_json
    from tts_component import TTSComponent, export_tts_manifest
    from data_preprocessing import DataPreprocessor, save_processed_data,␣
 ↪load_processed_data
    from lipsync_component import LipsyncGenerator,␣
 ↪HighResSpatioTemporalDiscriminator, LowResAudioVisualDiscriminator
    print(" Custom components imported successfully")
except ImportError as e:
    print(f" Error importing custom components: {e}")
    print("\nPlease ensure:")
    if IS_KAGGLE:
        print(f"  - Kaggle dataset at: {KAGGLE_COMPONENTS_PATH}")
        print("  - Contains: asr_component.py, mt_component.py, tts_component.
 ↪py, data_preprocessing.py, lipsync_component.py")
    else:
        print("  - Component files are in the same directory as this notebook")
        print("  - Or add the correct path using sys.path.append('path/to/
 ↪components')")

# ============================================================================
# IMPORT TTS LIBRARY (for Fine-tuning)
```

```
# ============================================================================
try:
    from TTS.config import BaseAudioConfig, BaseDatasetConfig
    from TTS.trainer import Trainer, TrainerArgs
    from TTS.tts.configs.shared_configs import BaseTTSConfig
    from TTS.tts.configs.tacotron2_config import Tacotron2Config
    from TTS.tts.datasets import BaseDatasetManager
    from TTS.tts.models.tacotron2 import Tacotron2
    from TTS.utils.audio import AudioProcessor
    print(" Coqui TTS library imported successfully")
except ImportError:
    print(" Warning: Coqui TTS library not fully found or installed.")
    print("  TTS Fine-tuning section might not work.")
    print("  Install with: pip install TTS")

# ============================================================================
# SET RANDOM SEEDS
# ============================================================================
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

print("\n" + "="*60)
print(" Setup and Imports completed")
print("="*60)
```

2 2. Configuration

Configure paths and parameters for the video dubbing pipeline.

[ ]:
```
# --- Kaggle Input Paths (Read-Only) ---
KAGGLE_VOXCELEB_ARCHIVE_DIR = Path("/kaggle/input/voxceleb2/VoxCeleb2")
KAGGLE_COMPONENT_DIR = Path("/kaggle/input/components/pytorch/default/1")

# --- Kaggle Working Paths (Writable) ---
KAGGLE_WORKING_DIR = Path("/kaggle/working")
EXTRACTED_DATASET_DIR = KAGGLE_WORKING_DIR / "voxceleb2_extracted"

# --- Base Directories ---
BASE_DIR = KAGGLE_WORKING_DIR
DATASET_DIR = EXTRACTED_DATASET_DIR  # Points to extracted data
PROCESSED_DATA_DIR = BASE_DIR / "processed_data"
MODEL_CHECKPOINT_DIR = BASE_DIR / "models"
```

```
OUTPUT_DIR = BASE_DIR / "output"
TTS_FINETUNE_OUTPUT_DIR = MODEL_CHECKPOINT_DIR / "tts_finetuned"
LIPSYNC_MODEL_DIR = MODEL_CHECKPOINT_DIR / "lipsync"

# Create output directories
for dir_path in [PROCESSED_DATA_DIR, MODEL_CHECKPOINT_DIR, OUTPUT_DIR,
                 TTS_FINETUNE_OUTPUT_DIR, LIPSYNC_MODEL_DIR]:
    dir_path.mkdir(parents=True, exist_ok=True)

print(f"{'='*60}")
print(f"{'='*60}")
print(f"Input Archive:       {KAGGLE_VOXCELEB_ARCHIVE_DIR}")
print(f"Components:          {KAGGLE_COMPONENT_DIR}")
print(f"Working Directory:   {KAGGLE_WORKING_DIR}")
print(f"Extracted Dataset:   {EXTRACTED_DATASET_DIR}")
print(f"Output Directory:    {OUTPUT_DIR}")
print(f"Model Checkpoints:   {MODEL_CHECKPOINT_DIR}")

# ============================================================================
# DATASET CONFIGURATION
# ============================================================================

# Check if archives exist
archive_files = []
if KAGGLE_VOXCELEB_ARCHIVE_DIR.exists():
    archive_files = list(KAGGLE_VOXCELEB_ARCHIVE_DIR.glob("vox2_dev_*"))
    print(f"\n Found {len(archive_files)} archive files in input")
else:
    print(f"\n Warning: Archive directory not found:␣
 ↪{KAGGLE_VOXCELEB_ARCHIVE_DIR}")

# Check if data is already extracted
is_extracted = EXTRACTED_DATASET_DIR.exists() and any(EXTRACTED_DATASET_DIR.
 ↪iterdir())

if is_extracted:
    print(f" Dataset already extracted to: {EXTRACTED_DATASET_DIR}")
    # Try to determine the structure
    dev_dirs = list(EXTRACTED_DATASET_DIR.glob("**/dev"))
    if dev_dirs:
        DEV_DATA_DIR = dev_dirs[0]
        print(f"  Found dev directory: {DEV_DATA_DIR}")
    else:
        DEV_DATA_DIR = EXTRACTED_DATASET_DIR
        print(f"  Using root as dev directory: {DEV_DATA_DIR}")
else:
    print(f"\n Dataset NOT extracted yet!")
```

```
    print(f"  Please run the extraction cell below before proceeding.")
    DEV_DATA_DIR = EXTRACTED_DATASET_DIR / "dev"  # Expected location after␣
 ↪extraction

# ============================================================================
# SPEAKER & VIDEO SELECTION
# ============================================================================

SPEAKER_ID = "id10270"  # Default speaker - will be updated after extraction

# Function to find first available video
def find_speaker_video(speaker_id, base_dir):
    """Find first video file for a given speaker"""
    speaker_paths = list(base_dir.rglob(f"**/{speaker_id}/*/*.mp4")) + \
                    list(base_dir.rglob(f"**/{speaker_id}/*/*.m4a"))
    return speaker_paths[0] if speaker_paths else None

# Try to find the target video
if is_extracted:
    TARGET_VIDEO_PATH = find_speaker_video(SPEAKER_ID, EXTRACTED_DATASET_DIR)
    if TARGET_VIDEO_PATH:
        print(f"\n Found video for {SPEAKER_ID}: {TARGET_VIDEO_PATH.name}")
    else:
        # Try to find any speaker
        all_videos = list(EXTRACTED_DATASET_DIR.rglob("**/*.mp4"))[:5]
        if all_videos:
            TARGET_VIDEO_PATH = all_videos[0]
            # Extract speaker ID from path
            parts = TARGET_VIDEO_PATH.parts
            for part in parts:
                if part.startswith("id"):
                    SPEAKER_ID = part
                    break
            print(f"\n Using first available video:")
            print(f"  Speaker: {SPEAKER_ID}")
            print(f"  Video: {TARGET_VIDEO_PATH.name}")
        else:
            TARGET_VIDEO_PATH = None
            print(f"\n No video files found in extracted data")
else:
    TARGET_VIDEO_PATH = None
    print(f"\n Target video will be selected after extraction")

# ============================================================================
# LANGUAGE SETTINGS
# ============================================================================
```

```
SOURCE_LANG = 'en'  # VoxCeleb2 is English
TARGET_LANG = 'es'  # Target language: 'es', 'fr', 'de', 'ru'

print(f"\n{'='*60}")
print("  Language Configuration")
print(f"{'='*60}")
print(f"Source: {SOURCE_LANG.upper()} → Target: {TARGET_LANG.upper()}")

# ============================================================================
# MODEL CONFIGURATIONS
# ============================================================================

ASR_MODEL_SIZE = "medium"  # Whisper: tiny, base, small, medium, large
MT_MODEL_NAME = "facebook/nllb-200-distilled-600M"  # NLLB model
BASE_TTS_MODEL_PATH = "tts_models/en/vctk/tacotron2-DDC"  # Coqui TTS base model

# ============================================================================
# TRAINING HYPERPARAMETERS
# ============================================================================
BATCH_SIZE = 16  # Reduced for Kaggle GPU memory
LEARNING_RATE_GEN = 5e-4
LEARNING_RATE_DISC = 1e-4

# Reduced iterations for Kaggle time limits (9 hours max)
TTS_FINETUNE_STEPS = 5000
LIPSYNC_PRETRAIN_STEPS = 5000  # Reduced from paper's 200k
LIPSYNC_FINETUNE_STEPS = 1000  # Reduced from paper's 10k

# ============================================================================
# AUDIO/VIDEO SETTINGS
# ============================================================================

SAMPLE_RATE = 16000
VIDEO_FPS = 25

# ============================================================================
# DEVICE CONFIGURATION
# ============================================================================

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

print(f"\n{'='*60}")
print("  Model & Training Configuration")
print(f"{'='*60}")
print(f"ASR Model:           Whisper-{ASR_MODEL_SIZE}")
print(f"MT Model:            {MT_MODEL_NAME.split('/')[-1]}")
print(f"TTS Base Model:      {BASE_TTS_MODEL_PATH.split('/')[-1]}")
```

```
print(f"\nTraining Parameters:")
print(f"  Batch Size:         {BATCH_SIZE}")
print(f"  Device:             {DEVICE}")
print(f"  TTS Steps:          {TTS_FINETUNE_STEPS}")
print(f"  Lipsync Pretrain:   {LIPSYNC_PRETRAIN_STEPS}")
print(f"  Lipsync Finetune:   {LIPSYNC_FINETUNE_STEPS}")

if DEVICE == "cuda":
    gpu_name = torch.cuda.get_device_name(0)
    gpu_memory = torch.cuda.get_device_properties(0).total_memory / 1e9
    print(f"\n GPU Available: {gpu_name} ({gpu_memory:.1f} GB)")
else:
    print(f"\n Running on CPU - training will be very slow!")

print(f"\n{'='*60}")
print(" Configuration Complete")
print(f"{'='*60}")

# ============================================================================
# SUMMARY & WARNINGS
# ============================================================================

if not is_extracted:
    print(f"\n{''*30}")
    print(" IMPORTANT: Dataset needs to be extracted!")
    print(" Please run the EXTRACTION CELL below before proceeding.")
    print(f"{''*30}")
elif TARGET_VIDEO_PATH is None:
    print(f"\n{''*30}")
    print(" WARNING: No target video found!")
    print(" Please verify the dataset structure after extraction.")
    print(f"{''*30}")
```

3 2.1 Extract VoxCeleb2 Dataset (Kaggle Only)

IMPORTANT: Run this cell ONCE to extract the VoxCeleb2 archives.

Note about file formats: - `vox2_dev_mp4_partaa` = Video files (MP4) **Best for video dubbing** - `vox2_dev_aac_partaa` = Audio only (AAC) - Use if MP4 not available

This may take 10-30 minutes depending on archive size.

[ ]:
```
# ============================================================================
# EXTRACT VOXCELEB2 DATASET
# ============================================================================

import tarfile
```

```
import zipfile
from tqdm.notebook import tqdm

# Check if already extracted
if EXTRACTED_DATASET_DIR.exists() and any(EXTRACTED_DATASET_DIR.iterdir()):
    print(" Dataset already extracted!")
    print(f"  Location: {EXTRACTED_DATASET_DIR}")

    # Show extracted content
    extracted_items = list(EXTRACTED_DATASET_DIR.iterdir())
    print(f"  Contents: {len(extracted_items)} items")
    for item in extracted_items[:5]:
        print(f"    - {item.name}")
else:
    print("Starting VoxCeleb2 extraction...")
    print(f"Source: {KAGGLE_VOXCELEB_ARCHIVE_DIR}")
    print(f"Destination: {EXTRACTED_DATASET_DIR}")

    # Create extraction directory
    EXTRACTED_DATASET_DIR.mkdir(parents=True, exist_ok=True)

    # Find archive files
    archive_files = sorted(KAGGLE_VOXCELEB_ARCHIVE_DIR.glob("vox2_dev_*"))

    if not archive_files:
        print(f"\n No archive files found in {KAGGLE_VOXCELEB_ARCHIVE_DIR}")
        print("Available files:")
        for f in KAGGLE_VOXCELEB_ARCHIVE_DIR.iterdir():
            print(f"  - {f.name}")
    else:
        print(f"\nFound {len(archive_files)} archive file(s):")
        for archive in archive_files:
            print(f"  - {archive.name}")

        # Extract each archive
        for archive_path in archive_files:
            print(f"\n{'='*60}")
            print(f"Extracting: {archive_path.name}")
            print(f"{'='*60}")

            try:
                # Determine archive type
                if archive_path.suffix in ['.tar', '.gz', '.tgz'] or 'tar' in␣
 ↪archive_path.name:
                    # TAR archive
                    print("Archive type: TAR")
                    with tarfile.open(archive_path, 'r:*') as tar:
```

```
                        members = tar.getmembers()
                        print(f"Total files: {len(members)}")

                        # Extract with progress bar
                        for member in tqdm(members, desc="Extracting"):
                            tar.extract(member, path=EXTRACTED_DATASET_DIR)

                elif archive_path.suffix == '.zip':
                    # ZIP archive
                    print("Archive type: ZIP")
                    with zipfile.ZipFile(archive_path, 'r') as zip_ref:
                        members = zip_ref.namelist()
                        print(f"Total files: {len(members)}")

                        # Extract with progress bar
                        for member in tqdm(members, desc="Extracting"):
                            zip_ref.extract(member, path=EXTRACTED_DATASET_DIR)

                else:
                    print(f" Unknown archive format: {archive_path.suffix}")
                    print("Attempting as TAR...")
                    # Try using shell command as fallback
                    import subprocess
                    result = subprocess.run(
                        ['tar', '-xf', str(archive_path), '-C',␣
↪str(EXTRACTED_DATASET_DIR)],
                        capture_output=True,
                        text=True
                    )
                    if result.returncode != 0:
                        print(f" Error: {result.stderr}")
                    else:
                        print(" Extraction successful (using tar command)")

                print(f" Completed: {archive_path.name}")

            except Exception as e:
                print(f" Error extracting {archive_path.name}: {e}")
                import traceback
                traceback.print_exc()

        print(f"\n{'='*60}")
        print(" Extraction Complete!")
        print(f"{'='*60}")

        # Show extracted structure
        if EXTRACTED_DATASET_DIR.exists():
```

```
            print(f"\nExtracted to: {EXTRACTED_DATASET_DIR}")
            extracted_items = list(EXTRACTED_DATASET_DIR.iterdir())
            print(f"Top-level items: {len(extracted_items)}")
            for item in extracted_items[:10]:
                item_type = "DIR" if item.is_dir() else "FILE"
                print(f"  [{item_type}] {item.name}")

            # Count total files
            all_files = list(EXTRACTED_DATASET_DIR.rglob("*.*"))
            print(f"\nTotal files extracted: {len(all_files)}")

            # Show video/audio files
            video_files = list(EXTRACTED_DATASET_DIR.rglob("*.mp4"))
            audio_files = list(EXTRACTED_DATASET_DIR.rglob("*.m4a")) + \
                          list(EXTRACTED_DATASET_DIR.rglob("*.aac"))
            print(f"  Video files (.mp4): {len(video_files)}")
            print(f"  Audio files (.m4a/.aac): {len(audio_files)}")

print(f"\n{'='*60}")
print(" Extraction Status Summary")
print(f"{'='*60}")
print(f"Extraction directory exists: {EXTRACTED_DATASET_DIR.exists()}")
if EXTRACTED_DATASET_DIR.exists():
    items = list(EXTRACTED_DATASET_DIR.iterdir())
    print(f"Items in directory: {len(items)}")
print(f"\n If extraction failed:")
print("  1. Check the archive file format in the Kaggle dataset")
print("  2. Manually inspect KAGGLE_VOXCELEB_ARCHIVE_DIR")
print("  3. Use shell commands: !tar -xf or !unzip")
```

[ ]:
```
# ============================================================================
# EXPLORE EXTRACTED DATASET STRUCTURE
# ============================================================================

if EXTRACTED_DATASET_DIR.exists() and any(EXTRACTED_DATASET_DIR.iterdir()):
    print(f"{'='*60}")
    print("  VoxCeleb2 Dataset Structure")
    print(f"{'='*60}\n")

    # Show directory tree (limited depth)
    def show_tree(directory, prefix="", max_depth=3, current_depth=0,␣
 ↪max_items=5):
        if current_depth >= max_depth:
            return

        try:
            items = sorted(directory.iterdir())[:max_items]
```

```
            for i, item in enumerate(items):
                is_last = (i == len(items) - 1)
                current_prefix = "   " if is_last else "   "
                next_prefix = "    " if is_last else "    "

                if item.is_dir():
                    subitem_count = len(list(item.iterdir()))
                    print(f"{prefix}{current_prefix} {item.name}/␣
↪({subitem_count} items)")
                    show_tree(item, prefix + next_prefix, max_depth,␣
↪current_depth + 1, max_items)
                else:
                    size_mb = item.stat().st_size / (1024**2)
                    print(f"{prefix}{current_prefix} {item.name} ({size_mb:.
↪2f} MB)")

            total_items = len(list(directory.iterdir()))
            if total_items > max_items:
                print(f"{prefix}    ... and {total_items - max_items} more␣
↪items")
        except Exception as e:
            print(f"{prefix}    Error: {e}")

    print(f"Root: {EXTRACTED_DATASET_DIR.name}/")
    show_tree(EXTRACTED_DATASET_DIR, max_depth=4, max_items=5)

    # Statistics
    print(f"\n{'='*60}")
    print(" Dataset Statistics")
    print(f"{'='*60}")

    # Count speakers
    speaker_dirs = [d for d in EXTRACTED_DATASET_DIR.rglob("*") if d.is_dir()␣
↪and d.name.startswith("id")]
    print(f"Total speakers found: {len(speaker_dirs)}")

    # Count files by type
    mp4_files = list(EXTRACTED_DATASET_DIR.rglob("*.mp4"))
    m4a_files = list(EXTRACTED_DATASET_DIR.rglob("*.m4a"))
    aac_files = list(EXTRACTED_DATASET_DIR.rglob("*.aac"))

    print(f"\nFile counts:")
    print(f"  Video (.mp4):  {len(mp4_files)}")
    print(f"  Audio (.m4a):  {len(m4a_files)}")
    print(f"  Audio (.aac):  {len(aac_files)}")
    print(f"  Total:         {len(mp4_files) + len(m4a_files) +␣
↪len(aac_files)}")
```

```
    # Show sample speaker
    if speaker_dirs:
        sample_speaker = speaker_dirs[0]
        video_files = list(sample_speaker.rglob("*.mp4")) + \
                      list(sample_speaker.rglob("*.m4a")) + \
                      list(sample_speaker.rglob("*.aac"))
        print(f"\nSample speaker: {sample_speaker.name}")
        print(f"  Total clips: {len(video_files)}")
        if video_files:
            sample_file = video_files[0]
            print(f"  Sample file: {sample_file.
 ↪relative_to(EXTRACTED_DATASET_DIR)}")

    # Show configured target
    print(f"\n{'='*60}")
    print(" Configured Target")
    print(f"{'='*60}")
    print(f"Speaker ID: {SPEAKER_ID}")
    if TARGET_VIDEO_PATH:
        print(f"Video path: {TARGET_VIDEO_PATH.
 ↪relative_to(KAGGLE_WORKING_DIR)}")
        print(f"File exists: {TARGET_VIDEO_PATH.exists()}")
    else:
        print("Video path: Not set - will be determined after extraction")

else:
    print(" Dataset not extracted yet!")
    print("Please run the extraction cell above first.")
```

4 3. Data Preparation

[ ]:
```
# ============================================================================
# 3. DATA PREPARATION (TARGET VIDEO)
# ============================================================================
print("="*60)
print(f"Preparing Target Video: {TARGET_VIDEO_PATH.name}")
print("="*60)

# --- Check if video file exists ---
if not TARGET_VIDEO_PATH.exists():
    print(f"ERROR: Target video file not found at: {TARGET_VIDEO_PATH}")
    print("Please ensure the TARGET_VIDEO_PATH in the Configuration cell is␣
 ↪correct.")

    target_video_processed_dir = None # Indicate failure
```

```
else:
    # --- Instantiate DataPreprocessor ---
    print("Initializing Data Preprocessor...")
    # Using default smoothing sigma=2.0 from the constructor
    preprocessor = DataPreprocessor(output_size=256)
    print(" Data Preprocessor initialized.")


    video_stem = TARGET_VIDEO_PATH.stem
    target_video_processed_dir = PROCESSED_DATA_DIR / SPEAKER_ID / video_stem
    target_video_processed_dir.mkdir(parents=True, exist_ok=True)
    print(f"Output directory for processed video data:␣
↪{target_video_processed_dir}")

    # --- Run Preprocessing ---
    # This will detect landmarks, align, crop, mask, and save frame-by-frame
    # inside the target_video_processed_dir
    print(f"\nStarting video processing for {TARGET_VIDEO_PATH.name}...")
    start_time = time.time()

    # The process_video function now saves data internally and returns metadata␣
↪dict
    processed_data_info = preprocessor.process_video(str(TARGET_VIDEO_PATH))

    end_time = time.time()

    if processed_data_info:
        print(f"\n Video processing complete for {TARGET_VIDEO_PATH.name}")
        print(f"  Total time: {end_time - start_time:.2f} seconds")
        print(f"  Processed {processed_data_info.get('frame_count', 'N/A')}␣
↪frames.")
        print(f"  Data saved in: {target_video_processed_dir}")

        # We store the directory path for later use
        target_processed_data_path = str(target_video_processed_dir)

        # Optional: Load back the metadata/arrays to verify
        # loaded_info = load_processed_data(target_processed_data_path)
        # if loaded_info:
        #     print("\nVerification: Loaded back processed data info:")
        #     print(f"  FPS: {loaded_info.get('fps')}")
        #     print(f"  Num Landmarks arrays: {len(loaded_info.
↪get('landmarks_smoothed', []))}")

    else:
        print(f"ERROR: Video processing failed for {TARGET_VIDEO_PATH.name}")
        target_processed_data_path = None # Indicate failure
```

```
print("\nData Preparation (Target Video) section finished.")
```

5 4. ASR (Transcription)

[ ]:
```
# ================================================================================
# 4. ASR (TRANSCRIPTION)
# ================================================================================
print("="*60)
print(f"Starting ASR Transcription for: {TARGET_VIDEO_PATH.name}")
print("="*60)

# --- Check if target video exists (redundant but good practice) ---
if not TARGET_VIDEO_PATH.exists():
    print(f"ERROR: Target video file not found at: {TARGET_VIDEO_PATH}")
    print("Please ensure the TARGET_VIDEO_PATH in the Configuration cell is␣
 ↪correct.")
    transcription_result = None # Indicate failure
else:
    # --- Instantiate ASR Component ---
    print("Initializing ASR Component...")
    try:
        asr = ASRComponent(model_size=ASR_MODEL_SIZE, device=DEVICE)
        print(" ASR Component initialized.")
    except Exception as e:
        print(f" ERROR: Failed to initialize ASR Component: {e}")
        asr = None # Indicate failure

    if asr:
        # --- Extract Audio ---
        print("\nExtracting audio from video...")
        audio_output_path = OUTPUT_DIR / f"{TARGET_VIDEO_PATH.stem}_audio.wav"
        try:
            extracted_audio_path = extract_audio_from_video(
                video_path=str(TARGET_VIDEO_PATH),
                output_path=str(audio_output_path),
                sample_rate=SAMPLE_RATE # Ensure 16kHz for Whisper
            )
            print(f" Audio extracted to: {extracted_audio_path}")
        except Exception as e:
            print(f" ERROR: Failed to extract audio: {e}")
            extracted_audio_path = None # Indicate failure

        # --- Run Transcription ---
        if extracted_audio_path and os.path.exists(extracted_audio_path):
            print("\nStarting transcription...")
```

```
            start_time = time.time()
            try:
                # Specify the source language if known, otherwise let Whisper␣
↪detect
                transcription_result = asr.transcribe(
                    audio_path=extracted_audio_path,
                    language=SOURCE_LANG, # Use None for auto-detect
                    verbose=False # Set to True for detailed Whisper progress
                )
                end_time = time.time()
                print(f"\n Transcription finished in {end_time - start_time:.
↪2f} seconds.")

                # --- Save Transcription ---
                transcript_json_path = OUTPUT_DIR / f"{TARGET_VIDEO_PATH.
↪stem}_transcript.json"
                transcript_srt_path = OUTPUT_DIR / f"{TARGET_VIDEO_PATH.
↪stem}_transcript.srt"

                try:
                    with open(transcript_json_path, 'w', encoding='utf-8') as f:
                        json.dump(transcription_result, f, ensure_ascii=False,␣
↪indent=2)
                    print(f"   Transcription saved to: {transcript_json_path}")

                    export_to_srt(transcription_result,␣
↪str(transcript_srt_path))
                    print(f"   SRT file saved to: {transcript_srt_path}")

                except Exception as e:
                    print(f"ERROR: Failed to save transcription files: {e}")

                # --- Optional: Print Summary ---
                print("\nTranscription Summary:")
                print_transcription(transcription_result, max_segments=5)

            except Exception as e:
                print(f"ERROR: Transcription failed: {e}")
                transcription_result = None # Indicate failure
        else:
            print("Skipping transcription because audio extraction failed or␣
↪file not found.")
            transcription_result = None # Indicate failure
    else:
        print("Skipping transcription because ASR component failed to␣
↪initialize.")
```

```
        transcription_result = None # Indicate failure

print("\nASR (Transcription) section finished.")
```

6 5. MT (Translation)

[ ]:
```
# ================================================================================
# 5. MT (TRANSLATION)
# ================================================================================
print("="*60)
print(f"Starting Machine Translation: {SOURCE_LANG.upper()} -> {TARGET_LANG.
 ↪upper()}")
print("="*60)

# --- Check if ASR results exist ---
if 'transcription_result' not in locals() or transcription_result is None:
    print(" ERROR: ASR transcription results not found or failed.")
    print("Please run the ASR cell (Cell 4) successfully first.")
    translated_segments = None # Indicate failure
elif not transcription_result.get('segments'):
    print(" ERROR: ASR transcription contains no segments to translate.")
    translated_segments = None # Indicate failure
else:
    # --- Instantiate MT Component ---
    print("Initializing MT Component...")
    try:
        mt = MTComponent(model_name=MT_MODEL_NAME, device=DEVICE)
        print(" MT Component initialized.")
    except Exception as e:
        print(f" ERROR: Failed to initialize MT Component: {e}")
        mt = None # Indicate failure

    if mt:
        # --- Run Translation ---
        print("\nTranslating ASR segments...")
        start_time = time.time()
        asr_segments = transcription_result['segments']

        try:
            # Use the batch-aware translate_segments method
            translated_segments = mt.translate_segments(
                segments=asr_segments,
                source_lang=SOURCE_LANG,
                target_lang=TARGET_LANG,
                verbose=True, # Show progress and warnings
                batch_size=16 #
```

```
            )
            end_time = time.time()
            print(f"\n Translation finished in {end_time - start_time:.2f}␣
 ↪seconds.")

            # --- Save Translation Results ---
            translation_json_path = OUTPUT_DIR / f"{TARGET_VIDEO_PATH.
 ↪stem}_translation_{TARGET_LANG}.json"
            try:
                # Use the utility function to save with metadata
                export_translation_to_json(
                    segments=translated_segments,
                    output_path=str(translation_json_path),
                    include_metadata=True
                )
                print(f"   Translated segments saved to:␣
 ↪{translation_json_path}")
            except Exception as e:
                print(f"ERROR: Failed to save translation JSON: {e}")

            # --- Optional: Print Summary ---
            if translated_segments:
                 print("\nTranslation Summary:")
                 print_translation_summary(translated_segments)

        except Exception as e:
            print(f"ERROR: Translation failed: {e}")
            translated_segments = None # Indicate failure
    else:
        print("Skipping translation because MT component failed to initialize.")
        translated_segments = None # Indicate failure

print("\nMT (Translation) section finished.")
```

7 6. TTS Fine-tuning (Stage 2)

[ ]:
```
# ==============================================================================
# 6. TTS FINE-TUNING (STAGE 2 - SPEAKER ADAPTATION)
# ==============================================================================
print("="*60)
print(f"Starting TTS Fine-tuning for Speaker: {SPEAKER_ID}")
print("="*60)

# --- Define Paths ---
speaker_audio_dir = DATASET_DIR / "dev" / "aac" / SPEAKER_ID #
fine_tune_data_dir = PROCESSED_DATA_DIR / SPEAKER_ID / "tts_finetune_data"
```

```
fine_tune_output_dir = TTS_FINETUNE_OUTPUT_DIR / SPEAKER_ID # Where the final␣
 ↪model will be saved

fine_tune_data_dir.mkdir(parents=True, exist_ok=True)
fine_tune_output_dir.mkdir(parents=True, exist_ok=True)

# --- 1. Data Preparation ---
print("\n--- 1. Preparing Fine-tuning Data ---")

# --- 1a. Select Audio Files (Aim for ~10-15 minutes) ---
target_duration_seconds = 10 * 60 # 10 minutes
selected_audio_files = []
total_duration = 0

print(f"Searching for audio files in: {speaker_audio_dir}")
if not speaker_audio_dir.exists():
    print(f" ERROR: Speaker audio directory not found: {speaker_audio_dir}")
    # Handle error appropriately - maybe stop or raise
else:
    # Find all audio files (e.g., .m4a, .wav, .mp3)
    potential_files = list(speaker_audio_dir.rglob("*.m4a"))
    random.shuffle(potential_files) # Shuffle to get a random sample
    estimated_clip_duration = 5
    num_clips_needed = int(target_duration_seconds / estimated_clip_duration)

    selected_audio_files = potential_files[:num_clips_needed]
    # A more robust way would load each file and check duration until target is␣
 ↪met
    total_duration = len(selected_audio_files) * estimated_clip_duration #␣
 ↪Rough estimate

    print(f"Selected {len(selected_audio_files)} audio files (estimated␣
 ↪~{total_duration/60:.1f} minutes).")

    # --- 1b. Convert/Copy Audio to WAV (Required by TTS trainer) ---
    wav_dir = fine_tune_data_dir / "wavs"
    wav_dir.mkdir(parents=True, exist_ok=True)
    processed_audio_paths = []

    print(f"Converting/Copying selected audio to WAV format in: {wav_dir}")
    for i, audio_file in enumerate(tqdm(selected_audio_files, desc="Processing␣
 ↪Audio")):
        output_wav_path = wav_dir / f"{audio_file.stem}.wav"
        try:
            # Use ffmpeg for robust conversion
            cmd = [
                'ffmpeg', '-i', str(audio_file),
```

```
                '-vn', '-acodec', 'pcm_s16le', '-ar', str(SAMPLE_RATE), '-ac',␣
↪'1',
                '-y', # Overwrite
                str(output_wav_path)
            ]
            subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,␣
↪stderr=subprocess.PIPE)
            processed_audio_paths.append(output_wav_path)
        except Exception as e:
            print(f"\nWarning: Failed to convert {audio_file}: {e}")
            if isinstance(e, subprocess.CalledProcessError):
                print(f"FFMPEG Error: {e.stderr.decode()}")


    print(f" Prepared {len(processed_audio_paths)} WAV files.")

    # --- 1c. Transcribe Audio Files ---
    if 'asr' not in locals() or asr is None:
        print("Initializing ASR Component for fine-tuning data...")
        try:
            asr = ASRComponent(model_size=ASR_MODEL_SIZE, device=DEVICE)
        except Exception as e:
            print(f" ERROR: Failed to initialize ASR: {e}")
            asr = None # Indicate failure

    transcripts = {}
    if asr and processed_audio_paths:
        print("\nTranscribing audio files for fine-tuning dataset...")
        # Use batch_transcribe (without saving individual JSONs here)
        # Ensure paths are strings
        paths_to_transcribe = [str(p) for p in processed_audio_paths]

        batch_results = asr.batch_transcribe(
            audio_paths=paths_to_transcribe,
            language=SOURCE_LANG # Assume source language
        )

        valid_transcripts_count = 0
        for result in batch_results:
            if result['success']:
                # Use relative path for metadata
                relative_path = Path(result['file']).
↪relative_to(fine_tune_data_dir)
                # Basic cleaning:
                clean_text = result['transcription']['text'].strip().lower()
                # Remove common punctuation, but check if base model expects it
```

```
                # clean_text = ''.join(c for c in clean_text if c.isalnum() or␣
↪c.isspace())
                transcripts[str(relative_path)] = clean_text
                valid_transcripts_count += 1
            else:
                print(f"Warning: Transcription failed for {result['file']}:␣
↪{result['error']}")
        print(f" Transcribed {valid_transcripts_count}/
↪{len(processed_audio_paths)} files.")
    else:
        print(" ERROR: ASR component not available or no audio files processed.␣
↪Cannot generate transcripts.")

    # --- 1d. Format Metadata (Coqui TTS LJSpeech-like format) ---
    metadata_path = fine_tune_data_dir / "metadata.csv"
    lines_written = 0
    if transcripts:
        print(f"\nWriting metadata file to: {metadata_path}")
        with open(metadata_path, 'w', encoding='utf-8') as f:
            for wav_path, text in transcripts.items():
                if text: # Ensure text is not empty
                    # Format: relative_path|transcript
                    f.write(f"{wav_path}|{text}\n")
                    lines_written += 1
        print(f" Metadata file created with {lines_written} entries.")
    else:
        print(" ERROR: No valid transcripts generated. Cannot create metadata␣
↪file.")


# --- 2. Training Configuration ---
print("\n--- 2. Configuring Fine-tuning ---")

# Check if data preparation was successful
if lines_written > 0:
    try:
        # --- 2a. Load Base Model Config ---
        # Initialize config object from the base model identifier
        config = Tacotron2Config()

        # Load config attributes from the base model's config file if possible
        # This requires Coqui TTS to find the base model
        try:
             # This automatically finds and loads the config if model name is␣
↪known to TTS
```

```
            base_model_config = BaseTTSConfig.init_from_url(BASE_TTS_MODEL_PATH␣
↪+ "/config.json")
            config.update(base_model_config)
            print(f"Loaded and updated config from base model:␣
↪{BASE_TTS_MODEL_PATH}")
        except Exception as e:
            print(f"Warning: Could not automatically load base config from␣
↪{BASE_TTS_MODEL_PATH}. Using default Tacotron2Config. Error: {e}")
            # Manually set critical audio parameters if defaults are wrong
            config.audio = BaseAudioConfig(
                sample_rate=SAMPLE_RATE, # Ensure consistency
                # Other params like n_fft, hop_length etc. should ideally match␣
↪base model
            )

        # --- 2b. Modify Config for Fine-tuning ---
        config.run_name = f"finetune_tacotron2_{SPEAKER_ID}"
        config.output_path = str(fine_tune_output_dir)

        # Dataset configuration
        config.datasets = [
            BaseDatasetConfig(
                name="my_speaker_dataset",
                meta_file_train="metadata.csv",
                path=str(fine_tune_data_dir)
            )
        ]
        config.num_loader_workers = 4 #3
        config.num_val_loader_workers = 2

        # Training parameters
        config.batch_size = max(16, BATCH_SIZE // 4) # Reduce batch size for␣
↪single GPU fine-tuning
        config.eval_batch_size = max(8, config.batch_size // 2)
        config.epochs = 1000 # Train for a fixed number of steps instead
        config.max_train_steps = TTS_FINETUNE_STEPS
        config.lr = 1e-4 # Fine-tuning often uses a smaller LR
        config.grad_clip = 1.0
        config.scheduler_after_epoch = False # Step scheduler per step

        # Checkpointing
        config.save_step = 1000 # Save checkpoints every N steps
        config.save_best_after = 1000 # Start saving best model after N steps
        config.keep_ckpts = 3 # Keep last 3 checkpoints + best model

        # Ensure correct sample rate is set
        config.audio.sample_rate = SAMPLE_RATE
```

```
        # --- 2c. Initialize Audio Processor ---
        ap = AudioProcessor.init_from_config(config)

        # --- 2d. Initialize Data Loader ---
        dm = BaseDatasetManager(config, ap)

        # --- 2e. Initialize Model ---
        model = Tacotron2.init_from_config(config,␣
 ↪pretrained_path=BASE_TTS_MODEL_PATH)

        print(f" Fine-tuning configured. Output will be saved to: {config.
 ↪output_path}")
        config_ready = True

    except Exception as e:
        print(f" ERROR: Failed to configure TTS training: {e}")
        config_ready = False
else:
    print("Skipping configuration due to data preparation errors.")
    config_ready = False

# --- 3. Run Fine-tuning ---
print("\n--- 3. Running Fine-tuning ---")

if config_ready:
    try:
        # --- Initialize Trainer ---
        trainer_args = TrainerArgs()
        trainer = Trainer(
            trainer_args, config, config.output_path, model=model,␣
 ↪train_samples=dm.train_dataloader(), eval_samples=dm.eval_dataloader()
        )

        # --- Start Training ---
        print(f"Starting fine-tuning for {TTS_FINETUNE_STEPS} steps...")
        start_time = time.time()
        trainer.fit()
        end_time = time.time()
        print(f"\n Fine-tuning complete in {(end_time - start_time)/60:.2f}␣
 ↪minutes.")
        print(f"  Fine-tuned model saved in: {config.output_path}")

        # Store the path for the next step
        fine_tuned_tts_model_path = config.output_path

    except Exception as e:
```

```
        print(f" ERROR: Fine-tuning process failed: {e}")
        fine_tuned_tts_model_path = None # Indicate failure
else:
    print("Skipping fine-tuning due to configuration or data preparation errors.
 ↪")
    fine_tuned_tts_model_path = None # Indicate failure

print("\nTTS Fine-tuning section finished.")
```

8 7. TTS Synthesis (Using Fine-tuned Model)

[ ]:
```
# ==============================================================================
# 7. TTS SYNTHESIS (USING FINE-TUNED MODEL)
# ==============================================================================
print("="*60)
print(f"Starting TTS Synthesis using fine-tuned model for Speaker:␣
 ↪{SPEAKER_ID}")
print("="*60)

# --- Check Prerequisites ---
if 'fine_tuned_tts_model_path' not in locals() or fine_tuned_tts_model_path is␣
 ↪None:
    print(" ERROR: Fine-tuned TTS model path not found or fine-tuning failed.")
    print("Please run the TTS Fine-tuning cell (Cell 6) successfully first.")
    synthesized_segments_info = None # Indicate failure
elif 'translated_segments' not in locals() or translated_segments is None:
    print(" ERROR: Translated text segments not found or translation failed.")
    print("Please run the MT cell (Cell 5) successfully first.")
    synthesized_segments_info = None # Indicate failure
else:
    # --- Instantiate TTS Component ---
    print("Initializing TTS Component...")
    try:
        # Instantiate the component (it loads models on demand)
        tts = TTSComponent(device=DEVICE)
        print(" TTS Component initialized.")
    except Exception as e:
        print(f" ERROR: Failed to initialize TTS Component: {e}")
        tts = None # Indicate failure

    if tts:
        # --- Define Output Directory for Synthesized Audio ---
        synthesis_output_dir = OUTPUT_DIR / f"{TARGET_VIDEO_PATH.
 ↪stem}_synthesized_audio_{TARGET_LANG}"
        synthesis_output_dir.mkdir(parents=True, exist_ok=True)
        print(f"Synthesized audio will be saved in: {synthesis_output_dir}")
```

```
        # --- Run Synthesis ---
        print("\nSynthesizing translated segments...")
        start_time = time.time()
        try:
            #
            synthesized_segments_info = tts.synthesize_segments(
                segments=translated_segments, # From Cell 5
                speaker_model_dir=str(fine_tuned_tts_model_path), # From Cell 6
                output_dir=str(synthesis_output_dir)
            )
            end_time = time.time()

            # Count successful syntheses
            successful_count = sum(1 for seg in synthesized_segments_info if␣
 ↪seg.get('audio_path'))
            print(f"\n Synthesis finished in {end_time - start_time:.2f}␣
 ↪seconds.")
            print(f"  Successfully synthesized {successful_count}/
 ↪{len(synthesized_segments_info)} segments.")

            # --- Save TTS Manifest ---
            manifest_path = OUTPUT_DIR / f"{TARGET_VIDEO_PATH.
 ↪stem}_tts_manifest_{TARGET_LANG}.json"
            try:
                export_tts_manifest(synthesized_segments_info,␣
 ↪str(manifest_path))
                print(f"   TTS Manifest saved to: {manifest_path}")
            except Exception as e:
                print(f" ERROR: Failed to save TTS manifest: {e}")

        except Exception as e:
            print(f" ERROR: TTS Synthesis failed: {e}")
            synthesized_segments_info = None # Indicate failure
    else:
        print("Skipping synthesis because TTS component failed to initialize.")
        synthesized_segments_info = None # Indicate failure

print("\nTTS Synthesis section finished.")
```

9 8. Lipsync Model Definition & Setup

[ ]:
```
# ==============================================================================
# 8. LIPSYNC MODEL DEFINITION & SETUP
# ==============================================================================
print("="*60)
```

```
print("Defining Lipsync Models, Optimizers, and Loss Functions")
print("="*60)

# --- Check Prerequisites ---
# No specific data prerequisites for this cell, but ensure imports are correct
if 'LipsyncGenerator' not in locals():
    print(" ERROR: Lipsync model components not imported correctly.")
    # Add necessary error handling or stop execution
else:
    # --- 1. Instantiate Models ---
    print("Instantiating Lipsync models...")
    try:
        # Generator (U-Net)
        generator = LipsyncGenerator().to(DEVICE)

        # Discriminators
        discriminator_highres = HighResSpatioTemporalDiscriminator().to(DEVICE)
        discriminator_lowres_audio = LowResAudioVisualDiscriminator().to(DEVICE)

        print(f" Generator instantiated on {DEVICE}.")
        print(f" Discriminator (HighRes) instantiated on {DEVICE}.")
        print(f" Discriminator (LowRes+Audio) instantiated on {DEVICE}.")

        # Optional: Print model summaries or parameter counts
        gen_params = sum(p.numel() for p in generator.parameters()) / 1e6
        disc_hr_params = sum(p.numel() for p in discriminator_highres.
 ↪parameters()) / 1e6
        disc_lr_params = sum(p.numel() for p in discriminator_lowres_audio.
 ↪parameters()) / 1e6
        print(f"  Generator Parameters: {gen_params:.2f} M")
        print(f"  Disc HighRes Params: {disc_hr_params:.2f} M")
        print(f"  Disc LowRes Params: {disc_lr_params:.2f} M")
        models_instantiated = True

    except Exception as e:
        print(f" ERROR: Failed to instantiate models: {e}")
        models_instantiated = False

    if models_instantiated:
        # --- 2. Define Optimizers ---
        print("\nDefining Optimizers...")
        try:
            # Adam optimizer as per paper
            optimizer_G = optim.Adam(generator.parameters(),␣
 ↪lr=LEARNING_RATE_GEN, betas=(0.5, 0.999))
            optimizer_D_highres = optim.Adam(discriminator_highres.
 ↪parameters(), lr=LEARNING_RATE_DISC, betas=(0.5, 0.999))
```

```
            optimizer_D_lowres = optim.Adam(discriminator_lowres_audio.
↪parameters(), lr=LEARNING_RATE_DISC, betas=(0.5, 0.999))

            print(f" Generator Optimizer: Adam (LR={LEARNING_RATE_GEN})")
            print(f" Discriminator Optimizers: Adam (LR={LEARNING_RATE_DISC})")
            optimizers_defined = True

        except Exception as e:
            print(f" ERROR: Failed to define optimizers: {e}")
            optimizers_defined = False

        # --- 3. Define Loss Functions ---
        print("\nDefining Loss Functions...")
        try:
            # 3a. Reconstruction Loss (LRec = alpha*MS-SSIM + (1-alpha)*L1)
            try:
                # Requires pytorch_msssim: pip install pytorch-msssim
                from pytorch_msssim import MS_SSIM

                # MS-SSIM expects specific range (e.g., 0-1 or -1 to 1) and␣
↪data format
                # data_range=1.0 assumes input images are normalized to [0, 1]
                # size_average=True returns the mean loss over the batch
                ms_ssim_loss = MS_SSIM(data_range=1.0, size_average=True,␣
↪channel=3)

                # MS-SSIM measures similarity (higher is better), so loss is 1␣
↪- MS_SSIM
                def loss_ms_ssim(pred, target):
                    return 1.0 - ms_ssim_loss(pred, target)

                print(" MS-SSIM Loss function loaded.")
            except ImportError:
                print(" WARNING: pytorch_msssim not found. Reconstruction loss␣
↪will only use L1.")
                print("          Install with: pip install pytorch-msssim")
                def loss_ms_ssim(pred, target):
                    return torch.tensor(0.0).to(pred.device) # Return 0 if␣
↪MS-SSIM is unavailable

            l1_loss = nn.L1Loss()

            ALPHA_MS_SSIM = 0.86 # From paper

            def reconstruction_loss(pred_img, target_img):
                l1 = l1_loss(pred_img, target_img)
```

```
                ms_ssim = loss_ms_ssim(pred_img, target_img)
                return (ALPHA_MS_SSIM * ms_ssim) + ((1 - ALPHA_MS_SSIM) * l1)

            # 3b. Landmark Loss (LLand = L2)
            landmark_loss = nn.MSELoss() # L2 Loss

            # 3c. GAN Loss (Hinge Loss) [cite: 328, 329]
            def loss_hinge_discriminator(disc_real, disc_fake):
                loss_real = torch.mean(F.relu(1. - disc_real))
                loss_fake = torch.mean(F.relu(1. + disc_fake))
                return loss_real + loss_fake

            def loss_hinge_generator(disc_fake):
                # Maximize D(G(z)) <-> Minimize -D(G(z))
                return -torch.mean(disc_fake)

            # 3d. Loss Weights (from paper)
            lambda_rec = 1.0
            lambda_land = 100.0
            lambda_gan = 1e-4

            print(" Reconstruction Loss (MS-SSIM + L1) defined.")
            print(" Landmark Loss (L2/MSE) defined.")
            print(" GAN Hinge Loss defined.")
            print(f" Loss Weights: Rec={lambda_rec}, Land={lambda_land},␣
↪GAN={lambda_gan}")
            losses_defined = True

        except Exception as e:
            print(f" ERROR: Failed to define loss functions: {e}")
            losses_defined = False

print("\nLipsync Model Definition & Setup section finished.")
```

10 9. Lipsync Training (Stage 1 - Multi-speaker)

[ ]:
```
# ==============================================================================
# 9. LIPSYNC TRAINING (STAGE 1 - MULTI-SPEAKER)
# ==============================================================================
print("="*60)
print("Starting Lipsync Stage 1 Training (Multi-speaker)")
print("="*60)

# --- Configuration ---
stage1_checkpoint_path = LIPSYNC_MODEL_DIR / "stage1_latest.pth"
stage1_best_checkpoint_path = LIPSYNC_MODEL_DIR / "stage1_best.pth"
```

```
num_train_steps = LIPSYNC_PRETRAIN_STEPS
save_checkpoint_freq = 5000 # Save checkpoint every N steps
log_freq = 100 # Print losses every N steps
num_reference_frames = 10 # N=10 as per paper/user notes
sequence_length = 9 # T=9 frames per training sample

# --- Check if models, optimizers, losses are defined ---
if ('generator' not in locals() or
    'discriminator_highres' not in locals() or
    'discriminator_lowres_audio' not in locals() or
    'optimizer_G' not in locals() or
    'optimizer_D_highres' not in locals() or
    'optimizer_D_lowres' not in locals() or
    'reconstruction_loss' not in locals() or
    'landmark_loss' not in locals() or
    'loss_hinge_discriminator' not in locals() or
    'loss_hinge_generator' not in locals()):
    print(" ERROR: Models, optimizers, or loss functions not defined.")
    print("Please run Cell 8 successfully first.")
    # Stop execution or handle error
else:
    # --- 1. Dataset and DataLoader ---
    print("\n--- 1. Setting up Dataset and DataLoader ---")

    class LipsyncDataset(Dataset):
        def __init__(self, data_root_dir, sequence_length=9, num_refs=10):
            self.data_root_dir = Path(data_root_dir)
            self.sequence_length = sequence_length
            self.num_refs = num_refs
            self.sequences = self._find_sequences() # Placeholder
            if not self.sequences:
                raise FileNotFoundError(f"No processed sequences found in␣
 ↪{data_root_dir}")

        def _find_sequences(self):
            seq_sources = []
            for speaker_dir in tqdm(list(self.data_root_dir.iterdir()),␣
 ↪desc="Scanning dataset"):
                if speaker_dir.is_dir():
                    for video_dir in speaker_dir.iterdir():
                        if video_dir.is_dir() and (video_dir / f"{video_dir.
 ↪stem}_meta.json").exists():
                            meta_path = video_dir / f"{video_dir.stem}_meta.
 ↪json"
                            with open(meta_path, 'r') as f:
                                meta = json.load(f)
```

```
                            if meta.get('num_frames', 0) >= self.
↪sequence_length:
                                seq_sources.append(video_dir)
        print(f"Found {len(seq_sources)} potential video sequence sources.")
        return seq_sources

    def __len__(self):
        #
        return len(self.sequences) * 50 #

    def __getitem__(self, idx):
        # Select a video source
        video_dir = self.sequences[idx % len(self.sequences)]
        meta_path = video_dir / f"{video_dir.stem}_meta.json"
        with open(meta_path, 'r') as f:
            meta = json.load(f)
        num_frames = meta['num_frames']

        # --- Sample a subsequence ---
        start_frame = random.randint(0, num_frames - self.sequence_length)
        frame_indices = list(range(start_frame, start_frame + self.
↪sequence_length))

        # --- Load data for the subsequence ---
        masked_frames_seq = []
        gt_frames_seq = []
        landmarks_seq = []
        audio_mels_seq = [] #

        # Load actual landmarks from npz
        npz_path = video_dir / f"{video_dir.stem}_data.npz"
        np_data = np.load(npz_path, allow_pickle=True)
        all_landmarks = np_data['landmarks_smoothed'] # object array

        for i in frame_indices:
            # Load masked frame
            mf_path = video_dir / "masked_inputs" / f"frame_{i:06d}.png"
            img = cv2.imread(str(mf_path))
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # BGR -> RGB
            img = img.astype(np.float32) / 255.0 # Normalize 0-1
            masked_frames_seq.append(torch.tensor(img).permute(2, 0, 1)) #␣
↪HWC -> CHW

            # Load ground truth frame (cropped face)
            gt_path = video_dir / "cropped_faces" / f"frame_{i:06d}.png"
            gt_img = cv2.imread(str(gt_path))
            gt_img = cv2.cvtColor(gt_img, cv2.COLOR_BGR2RGB)
```

```
                gt_img = gt_img.astype(np.float32) / 255.0
                gt_frames_seq.append(torch.tensor(gt_img).permute(2, 0, 1))

                landmarks = all_landmarks[i]
                if landmarks is None: #
                     landmarks = np.zeros((13, 2), dtype=np.float32) #␣
↪Placeholder for 13 landmarks
                     # Or load the full 478 landmarks and select 13
                else:

                    landmarks = landmarks[:13, :2] # Take first 13, x,y

                # Normalize landmarks (e.g., relative to image size 256)
                landmarks = (landmarks / 256.0) * 2.0 - 1.0 # Normalize to [-1,␣
↪1]
                landmarks_seq.append(torch.tensor(landmarks, dtype=torch.
↪float32))


                audio_mel = torch.randn(64, 24) # Dummy data [C_aud, A_len]
                audio_mels_seq.append(audio_mel)

            # --- Select Reference Frames ---
            # Paper uses K-Means on landmarks, simplified: sample randomly
            ref_indices = random.sample([k for k in range(num_frames) if k not␣
↪in frame_indices], self.num_refs)
            ref_frames_seq = []
            for i in ref_indices:
                ref_path = video_dir / "reference_masks" / f"frame_{i:06d}.png"
                ref_img = cv2.imread(str(ref_path))
                ref_img = cv2.cvtColor(ref_img, cv2.COLOR_BGR2RGB)
                ref_img = ref_img.astype(np.float32) / 255.0
                ref_frames_seq.append(torch.tensor(ref_img).permute(2, 0, 1))

            # Stack sequences into tensors
            masked_frames = torch.stack(masked_frames_seq) # [T, C, H, W]
            gt_frames = torch.stack(gt_frames_seq)         # [T, C, H, W]
            gt_landmarks = torch.stack(landmarks_seq)       # [T, 13, 2]
            audio_mels = torch.stack(audio_mels_seq)       # [T, C_aud, A_len]
            ref_frames = torch.stack(ref_frames_seq)       # [N, C, H, W]

            return masked_frames, audio_mels, ref_frames, gt_frames,␣
↪gt_landmarks

    # --- Instantiate Dataset and DataLoader ---
    try:
```

```
        # !! Make sure PROCESSED_DATA_DIR points to the root containing speaker␣
↪folders !!
        lipsync_dataset = LipsyncDataset(PROCESSED_DATA_DIR,␣
↪sequence_length=sequence_length, num_refs=num_reference_frames)
        # Pin memory for faster GPU transfer
        use_pin_memory = (DEVICE == 'cuda')
        data_loader = DataLoader(
            lipsync_dataset,
            batch_size=BATCH_SIZE,
            shuffle=True,
            num_workers=4, #
            pin_memory=use_pin_memory,
            drop_last=True # Important for consistent batch sizes
        )
        print(f" Dataset loaded. Estimated steps per epoch: {len(data_loader)}")
        dataloader_ready = True
    except Exception as e:
        print(f" ERROR: Failed to create Dataset/DataLoader: {e}")
        print(f"  Ensure preprocessed data exists in '{PROCESSED_DATA_DIR}'")
        dataloader_ready = False


    # --- 2. Load Checkpoint (Optional) ---
    start_step = 0
    best_loss_g = float('inf') # Track best generator loss for saving best model

    LOAD_CHECKPOINT = False # Set to True to resume training
    if LOAD_CHECKPOINT and stage1_checkpoint_path.exists():
        print(f"\n--- Loading Checkpoint: {stage1_checkpoint_path} ---")
        try:
            checkpoint = torch.load(stage1_checkpoint_path, map_location=DEVICE)
            generator.load_state_dict(checkpoint['generator_state_dict'])
            discriminator_highres.
↪load_state_dict(checkpoint['discriminator_highres_state_dict'])
            discriminator_lowres_audio.
↪load_state_dict(checkpoint['discriminator_lowres_audio_state_dict'])
            optimizer_G.load_state_dict(checkpoint['optimizer_G_state_dict'])
            optimizer_D_highres.
↪load_state_dict(checkpoint['optimizer_D_highres_state_dict'])
            optimizer_D_lowres.
↪load_state_dict(checkpoint['optimizer_D_lowres_state_dict'])
            start_step = checkpoint.get('step', 0) + 1
            best_loss_g = checkpoint.get('best_loss_g', float('inf'))
            print(f" Checkpoint loaded. Resuming from step {start_step}.")
        except Exception as e:
```

```
            print(f" ERROR: Failed to load checkpoint: {e}. Starting from␣
↪scratch.")
            start_step = 0
            best_loss_g = float('inf')
    else:
        print("\n--- Starting Training from Scratch ---")


    # --- 3. Training Loop ---
    if dataloader_ready:
        print(f"\n--- Starting Stage 1 Training Loop for {num_train_steps}␣
↪steps ---")
        generator.train()
        discriminator_highres.train()
        discriminator_lowres_audio.train()

        # Use tqdm for overall progress across steps, not epochs
        pbar = tqdm(total=num_train_steps, initial=start_step, desc="Stage 1␣
↪Training")
        data_iter = iter(data_loader)
        current_step = start_step

        while current_step < num_train_steps:
            # --- Get Data Batch ---
            try:
                masked_frames, audio_mels, ref_frames, gt_frames, gt_landmarks␣
↪= next(data_iter)
            except StopIteration:
                # Epoch finished, reset iterator
                data_iter = iter(data_loader)
                masked_frames, audio_mels, ref_frames, gt_frames, gt_landmarks␣
↪= next(data_iter)

            # Move data to device
            masked_frames = masked_frames.to(DEVICE) # [B, T, C, H, W]
            audio_mels = audio_mels.to(DEVICE)       # [B, T, C_aud, A_len]
            ref_frames = ref_frames.to(DEVICE)       # [B, N, C, H, W]
            gt_frames = gt_frames.to(DEVICE)         # [B, T, C, H, W]
            gt_landmarks = gt_landmarks.to(DEVICE)   # [B, T, 13, 2]

            # --- Train Discriminators ---
            optimizer_D_highres.zero_grad()
            optimizer_D_lowres.zero_grad()

            # Generate fake data
            with torch.no_grad():
                # pred_images: [B, T, C, H, W], pred_landmarks: [B, T, 13, 2]
```

```
                pred_images, _ = generator(masked_frames, audio_mels,␣
↪ref_frames)

            # --- HighRes Discriminator ---
            # Select 3 sequential frames randomly from the sequence T=9
            start_idx_hr = random.randint(0, sequence_length - 3)
            real_hr = gt_frames[:, start_idx_hr : start_idx_hr + 3] # [B, 3, C,␣
↪H, W]
            fake_hr = pred_images[:, start_idx_hr : start_idx_hr + 3].detach()␣
↪# Detach from generator graph

            disc_real_hr = discriminator_highres(real_hr)
            disc_fake_hr = discriminator_highres(fake_hr)
            loss_D_hr = loss_hinge_discriminator(disc_real_hr, disc_fake_hr)

            # --- LowRes+Audio Discriminator ---
            # Uses full sequence T=9
            real_lr = gt_frames # [B, T, C, H, W]
            fake_lr = pred_images.detach() # [B, T, C, H, W]
            audio_lr = audio_mels # [B, T, C_aud, A_len]

            disc_real_lr = discriminator_lowres_audio(real_lr, audio_lr)
            disc_fake_lr = discriminator_lowres_audio(fake_lr, audio_lr) #
            loss_D_lr = loss_hinge_discriminator(disc_real_lr, disc_fake_lr)

            # --- Update Discriminators ---
            loss_D = loss_D_hr + loss_D_lr
            loss_D.backward()
            # Apply gradient clipping (from paper)
            torch.nn.utils.clip_grad_norm_(discriminator_highres.parameters(),␣
↪10.0)
            torch.nn.utils.clip_grad_norm_(discriminator_lowres_audio.
↪parameters(), 10.0)
            optimizer_D_highres.step()
            optimizer_D_lowres.step()


            # --- Train Generator ---
            optimizer_G.zero_grad()

            # Generate fake data (re-run for generator training)
            pred_images, pred_landmarks = generator(masked_frames, audio_mels,␣
↪ref_frames)

            # 1. Reconstruction Loss (on full sequence)
            loss_rec = reconstruction_loss(pred_images, gt_frames)
```

```
            # 2. Landmark Loss (on full sequence)
            loss_land = landmark_loss(pred_landmarks, gt_landmarks)

            # 3. GAN Loss (HighRes)
            # Use the same 3 frames as discriminator
            fake_hr_g = pred_images[:, start_idx_hr : start_idx_hr + 3]
            disc_fake_hr_g = discriminator_highres(fake_hr_g)
            loss_G_gan_hr = loss_hinge_generator(disc_fake_hr_g)

            # 4. GAN Loss (LowRes+Audio)
            fake_lr_g = pred_images
            disc_fake_lr_g = discriminator_lowres_audio(fake_lr_g, audio_lr)
            loss_G_gan_lr = loss_hinge_generator(disc_fake_lr_g)

            # 5. Total Generator Loss
            loss_G_gan = loss_G_gan_hr + loss_G_gan_lr
            loss_G = (lambda_rec * loss_rec) + \
                     (lambda_land * loss_land) + \
                     (lambda_gan * loss_G_gan)

            # --- Update Generator ---
            loss_G.backward()
            # Apply gradient clipping (from paper)
            torch.nn.utils.clip_grad_norm_(generator.parameters(), 10.0)
            optimizer_G.step()

            # --- Logging ---
            if current_step % log_freq == 0:
                pbar.set_postfix({
                    "Loss_D": f"{loss_D.item():.4f}",
                    "Loss_G": f"{loss_G.item():.4f}",
                    "Loss_Rec": f"{loss_rec.item():.4f}",
                    "Loss_Land": f"{loss_land.item():.4f}",
                    "Loss_GAN": f"{loss_G_gan.item():.4f}"
                })

            # --- Checkpointing ---
            if current_step % save_checkpoint_freq == 0 or current_step ==␣
↪num_train_steps - 1:
                is_best = loss_G.item() < best_loss_g
                if is_best:
                    best_loss_g = loss_G.item()
                    print(f"\n  New best generator loss: {best_loss_g:.4f} at␣
↪step {current_step}")

                checkpoint_data = {
```

```
                    'step': current_step,
                    'generator_state_dict': generator.state_dict(),
                    'discriminator_highres_state_dict': discriminator_highres.
↪state_dict(),
                    'discriminator_lowres_audio_state_dict':␣
↪discriminator_lowres_audio.state_dict(),
                    'optimizer_G_state_dict': optimizer_G.state_dict(),
                    'optimizer_D_highres_state_dict': optimizer_D_highres.
↪state_dict(),
                    'optimizer_D_lowres_state_dict': optimizer_D_lowres.
↪state_dict(),
                    'loss_G': loss_G.item(),
                    'best_loss_g': best_loss_g,
                }
                # Save latest checkpoint
                torch.save(checkpoint_data, stage1_checkpoint_path)
                print(f"\n  Checkpoint saved to {stage1_checkpoint_path} at␣
↪step {current_step}")
                # Save best checkpoint
                if is_best:
                    torch.save(checkpoint_data, stage1_best_checkpoint_path)
                    print(f"  Best model checkpoint saved to␣
↪{stage1_best_checkpoint_path}")


            # Update progress bar and step counter
            pbar.update(1)
            current_step += 1

        pbar.close()
        print("\n Stage 1 Training Finished.")

        # Save final model explicitly
        final_checkpoint_data = {
            'step': current_step - 1,
            'generator_state_dict': generator.state_dict(),

        }
        final_model_path = LIPSYNC_MODEL_DIR / "stage1_final.pth"
        torch.save(final_checkpoint_data, final_model_path)
        print(f"  Final Stage 1 Generator saved to {final_model_path}")

    else:
        print("Skipping Stage 1 training due to DataLoader setup failure.")
```

```
print("\nLipsync Training (Stage 1 - Multi-speaker) section finished.")
```

11 10. Lipsync Fine-tuning (Stage 2 - Single-speaker)

[ ]:
```
# ========================================================================
# 10. LIPSYNC FINE-TUNING (STAGE 2 - SINGLE-SPEAKER)
# ========================================================================
print("="*60)
print(f"Starting Lipsync Stage 2 Fine-tuning for Speaker: {SPEAKER_ID}")
print("="*60)

# --- Configuration ---
stage1_checkpoint_to_load = stage1_best_checkpoint_path # Load the best model␣
 ↪from Stage 1
stage2_output_dir = LIPSYNC_MODEL_DIR / f"{SPEAKER_ID}_stage2"
stage2_output_dir.mkdir(parents=True, exist_ok=True)
stage2_checkpoint_path = stage2_output_dir / "stage2_latest.pth"
stage2_best_checkpoint_path = stage2_output_dir / "stage2_best.pth"
stage2_final_model_path = stage2_output_dir / "stage2_final_generator.pth"
num_finetune_steps = LIPSYNC_FINETUNE_STEPS

# --- Check Prerequisites ---
if not stage1_checkpoint_to_load.exists():
    print(f" ERROR: Stage 1 checkpoint not found at:␣
 ↪{stage1_checkpoint_to_load}")
    print("Please run Stage 1 training (Cell 9) successfully or provide a valid␣
 ↪path.")
    fine_tuning_possible = False
elif 'target_processed_data_path' not in locals() or target_processed_data_path␣
 ↪is None:
    print(f" ERROR: Processed data path for the target video not found.")
    print("Please run Data Preparation (Cell 3) successfully first.")
    fine_tuning_possible = False
elif ('generator' not in locals() or # Check if models etc. are still in memory
      'discriminator_highres' not in locals() or
      'optimizer_G' not in locals()):
     print(" ERROR: Models, optimizers, or losses not defined in memory.")
     print("Please re-run Cell 8.")
     fine_tuning_possible = False
else:
    fine_tuning_possible = True

if fine_tuning_possible:
    # --- 1. Load Stage 1 Generator Checkpoint ---
    print(f"\n--- Loading Stage 1 Generator Checkpoint:␣
 ↪{stage1_checkpoint_to_load} ---")
```

```
    try:
        checkpoint = torch.load(stage1_checkpoint_to_load, map_location=DEVICE)
        generator.load_state_dict(checkpoint['generator_state_dict'])
        discriminator_highres.
↪load_state_dict(checkpoint['discriminator_highres_state_dict'])
        discriminator_lowres_audio.
↪load_state_dict(checkpoint['discriminator_lowres_audio_state_dict'])
        optimizer_G = optim.Adam(generator.parameters(), lr=LEARNING_RATE_GEN,␣
↪betas=(0.5, 0.999))
        optimizer_D_highres = optim.Adam(discriminator_highres.parameters(),␣
↪lr=LEARNING_RATE_DISC, betas=(0.5, 0.999))
        optimizer_D_lowres = optim.Adam(discriminator_lowres_audio.
↪parameters(), lr=LEARNING_RATE_DISC, betas=(0.5, 0.999))

        print(f" Generator weights loaded from Stage 1 checkpoint.")
        print(f" Discriminators and Optimizers re-initialized.") # Or indicate␣
↪loading if done
        models_loaded = True
    except Exception as e:
        print(f" ERROR: Failed to load Stage 1 checkpoint: {e}")
        models_loaded = False

    # --- 2. Dataset and DataLoader (Speaker Specific) ---
    if models_loaded:
        print("\n--- 2. Setting up Speaker-Specific Dataset ---")
        try:
            class SpeakerLipsyncDataset(LipsyncDataset):
                def __init__(self, target_video_dir, sequence_length=9,␣
↪num_refs=10):
                    self.sequence_length = sequence_length
                    self.num_refs = num_refs
                    self.video_dir = Path(target_video_dir)
                    if not self.video_dir.is_dir():
                        raise FileNotFoundError(f"Target video data dir not␣
↪found: {target_video_dir}")

                    meta_path = self.video_dir / f"{self.video_dir.stem}_meta.
↪json"
                    if not meta_path.exists():
                        raise FileNotFoundError(f"Metadata file not found in␣
↪{target_video_dir}")

                    with open(meta_path, 'r') as f:
                        self.meta = json.load(f)
                    self.num_frames = self.meta['num_frames']
                    if self.num_frames < self.sequence_length:
```

```
                        raise ValueError(f"Video has only {self.num_frames}␣
↪frames, less than sequence length {self.sequence_length}")

                    npz_path = self.video_dir / f"{self.video_dir.stem}_data.
↪npz"
                    if not npz_path.exists():
                        raise FileNotFoundError(f"Data npz file not found in␣
↪{target_video_dir}")
                    self.np_data = np.load(npz_path, allow_pickle=True)
                    self.all_landmarks = self.np_data['landmarks_smoothed'] #␣
↪object array

                def __len__(self):
                    # Length is number of possible start frames
                    return self.num_frames - self.sequence_length + 1

                def __getitem__(self, idx):
                    # idx determines the start_frame directly
                    start_frame = idx
                    frame_indices = list(range(start_frame, start_frame + self.
↪sequence_length))

                    # --- Load data for the subsequence ---
                    # (Copied and adapted loading logic from Cell 9's dataset)
                    masked_frames_seq = []
                    gt_frames_seq = []
                    landmarks_seq = []
                    audio_mels_seq = []

                    for i in frame_indices:
                        mf_path = self.video_dir / "masked_inputs" / f"frame_{i:
↪06d}.png"
                        img = cv2.imread(str(mf_path))
                        if img is None: raise IOError(f"Failed to load masked␣
↪frame: {mf_path}")
                        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                        img = img.astype(np.float32) / 255.0
                        masked_frames_seq.append(torch.tensor(img).permute(2,␣
↪0, 1))

                        gt_path = self.video_dir / "cropped_faces" / f"frame_{i:
↪06d}.png"
                        gt_img = cv2.imread(str(gt_path))
                        if gt_img is None: raise IOError(f"Failed to load GT␣
↪frame: {gt_path}")
                        gt_img = cv2.cvtColor(gt_img, cv2.COLOR_BGR2RGB)
```

```
                        gt_img = gt_img.astype(np.float32) / 255.0
                        gt_frames_seq.append(torch.tensor(gt_img).permute(2, 0,␣
↪1))

                        landmarks = self.all_landmarks[i]
                        if landmarks is None or landmarks.size == 0:
                            landmarks = np.zeros((13, 2), dtype=np.float32)
                        else:
                            landmarks = landmarks[:13, :2] # Assuming first 13
                        landmarks = (landmarks / self.meta['output_size']) * 2.
↪0 - 1.0 # Normalize [-1, 1]
                        landmarks_seq.append(torch.tensor(landmarks,␣
↪dtype=torch.float32))

                        # !! Replace dummy audio with actual precomputed mels !!
                        audio_mel = torch.randn(64, 24) # Dummy data
                        audio_mels_seq.append(audio_mel)

                    # --- Select Reference Frames ---
                    valid_indices = [k for k in range(self.num_frames) if self.
↪all_landmarks[k] is not None] # Use frames where landmarks exist
                    available_ref_indices = [k for k in valid_indices if k not␣
↪in frame_indices]
                    if len(available_ref_indices) < self.num_refs:
                        # If not enough unique refs, sample with replacement␣
↪from valid ones
                        ref_indices = random.choices(valid_indices, k=self.
↪num_refs)
                    else:
                        ref_indices = random.sample(available_ref_indices, self.
↪num_refs)

                    ref_frames_seq = []
                    for i in ref_indices:
                        ref_path = self.video_dir / "reference_masks" /␣
↪f"frame_{i:06d}.png"
                        ref_img = cv2.imread(str(ref_path))
                        if ref_img is None: raise IOError(f"Failed to load ref␣
↪frame: {ref_path}")
                        ref_img = cv2.cvtColor(ref_img, cv2.COLOR_BGR2RGB)
                        ref_img = ref_img.astype(np.float32) / 255.0
                        ref_frames_seq.append(torch.tensor(ref_img).permute(2,␣
↪0, 1))

                    masked_frames = torch.stack(masked_frames_seq)
                    gt_frames = torch.stack(gt_frames_seq)
```

```
                    gt_landmarks = torch.stack(landmarks_seq)
                    audio_mels = torch.stack(audio_mels_seq)
                    ref_frames = torch.stack(ref_frames_seq)

                    return masked_frames, audio_mels, ref_frames, gt_frames,␣
↪gt_landmarks

            # Instantiate dataset using the path from Cell 3
            speaker_dataset = SpeakerLipsyncDataset(target_processed_data_path,␣
↪sequence_length=sequence_length, num_refs=num_reference_frames)

            # Use a smaller batch size suitable for fine-tuning / single GPU
            fine_tune_batch_size = max(4, BATCH_SIZE // 8)

            speaker_dataloader = DataLoader(
                speaker_dataset,
                batch_size=fine_tune_batch_size,
                shuffle=True,
                num_workers=2, # Usually lower is fine for smaller dataset
                pin_memory=use_pin_memory,
                drop_last=True
            )
            print(f" Speaker dataset loaded ({len(speaker_dataset)} samples).")
            print(f"  DataLoader steps per epoch: {len(speaker_dataloader)}")
            dataloader_ready = True
        except Exception as e:
            print(f" ERROR: Failed to create Speaker Dataset/DataLoader: {e}")
            dataloader_ready = False

    # --- 3. Fine-tuning Loop ---
    if dataloader_ready and models_loaded:
        print(f"\n--- Starting Stage 2 Fine-tuning Loop for␣
↪{num_finetune_steps} steps ---")
        generator.train()
        discriminator_highres.train()
        discriminator_lowres_audio.train()

        # Load latest stage 2 checkpoint if exists
        start_step_stage2 = 0
        best_loss_g_stage2 = float('inf')
        LOAD_STAGE2_CHECKPOINT = False # Set True to resume Stage 2
        if LOAD_STAGE2_CHECKPOINT and stage2_checkpoint_path.exists():
            print(f"\n--- Loading Stage 2 Checkpoint: {stage2_checkpoint_path}␣
↪---")
            try:
                checkpoint = torch.load(stage2_checkpoint_path,␣
↪map_location=DEVICE)
```

```
                # Load all states
                generator.load_state_dict(checkpoint['generator_state_dict'])
                discriminator_highres.
↪load_state_dict(checkpoint['discriminator_highres_state_dict'])
                discriminator_lowres_audio.
↪load_state_dict(checkpoint['discriminator_lowres_audio_state_dict'])
                optimizer_G.
↪load_state_dict(checkpoint['optimizer_G_state_dict'])
                optimizer_D_highres.
↪load_state_dict(checkpoint['optimizer_D_highres_state_dict'])
                optimizer_D_lowres.
↪load_state_dict(checkpoint['optimizer_D_lowres_state_dict'])
                start_step_stage2 = checkpoint.get('step', 0) + 1
                best_loss_g_stage2 = checkpoint.get('best_loss_g_stage2',␣
↪float('inf'))
                print(f" Stage 2 Checkpoint loaded. Resuming from step␣
↪{start_step_stage2}.")
            except Exception as e:
                print(f" ERROR loading Stage 2 checkpoint: {e}. Starting Stage␣
↪2 from scratch.")
                start_step_stage2 = 0
                best_loss_g_stage2 = float('inf')

        # Use tqdm for progress
        pbar_stage2 = tqdm(total=num_finetune_steps, initial=start_step_stage2,␣
↪desc=f"Stage 2 Fine-tune ({SPEAKER_ID})")
        data_iter_stage2 = iter(speaker_dataloader)
        current_step_stage2 = start_step_stage2

        while current_step_stage2 < num_finetune_steps:
            # --- Get Data Batch ---
            try:
                masked_frames, audio_mels, ref_frames, gt_frames, gt_landmarks␣
↪= next(data_iter_stage2)
            except StopIteration:
                # Epoch finished, reset iterator
                data_iter_stage2 = iter(speaker_dataloader)
                masked_frames, audio_mels, ref_frames, gt_frames, gt_landmarks␣
↪= next(data_iter_stage2)

            # Move data to device
            masked_frames = masked_frames.to(DEVICE)
            audio_mels = audio_mels.to(DEVICE)
            ref_frames = ref_frames.to(DEVICE)
            gt_frames = gt_frames.to(DEVICE)
            gt_landmarks = gt_landmarks.to(DEVICE)
```

```
            # --- Train Discriminators (Identical logic to Stage 1) ---
            optimizer_D_highres.zero_grad()
            optimizer_D_lowres.zero_grad()
            with torch.no_grad():
                pred_images, _ = generator(masked_frames, audio_mels,␣
↪ref_frames)

            # HighRes D
            start_idx_hr = random.randint(0, sequence_length - 3)
            real_hr = gt_frames[:, start_idx_hr : start_idx_hr + 3]
            fake_hr = pred_images[:, start_idx_hr : start_idx_hr + 3].detach()
            disc_real_hr = discriminator_highres(real_hr)
            disc_fake_hr = discriminator_highres(fake_hr)
            loss_D_hr = loss_hinge_discriminator(disc_real_hr, disc_fake_hr)

            # LowRes D
            real_lr, fake_lr, audio_lr = gt_frames, pred_images.detach(),␣
↪audio_mels
            disc_real_lr = discriminator_lowres_audio(real_lr, audio_lr)
            disc_fake_lr = discriminator_lowres_audio(fake_lr, audio_lr)
            loss_D_lr = loss_hinge_discriminator(disc_real_lr, disc_fake_lr)

            # Update D
            loss_D = loss_D_hr + loss_D_lr
            loss_D.backward()
            torch.nn.utils.clip_grad_norm_(discriminator_highres.parameters(),␣
↪10.0)
            torch.nn.utils.clip_grad_norm_(discriminator_lowres_audio.
↪parameters(), 10.0)
            optimizer_D_highres.step()
            optimizer_D_lowres.step()

            # --- Train Generator (Identical logic to Stage 1) ---
            optimizer_G.zero_grad()
            pred_images, pred_landmarks = generator(masked_frames, audio_mels,␣
↪ref_frames)

            loss_rec = reconstruction_loss(pred_images, gt_frames)
            loss_land = landmark_loss(pred_landmarks, gt_landmarks)

            fake_hr_g = pred_images[:, start_idx_hr : start_idx_hr + 3]
            disc_fake_hr_g = discriminator_highres(fake_hr_g)
            loss_G_gan_hr = loss_hinge_generator(disc_fake_hr_g)

            fake_lr_g = pred_images
            disc_fake_lr_g = discriminator_lowres_audio(fake_lr_g, audio_lr)
```

```
            loss_G_gan_lr = loss_hinge_generator(disc_fake_lr_g)

            loss_G_gan = loss_G_gan_hr + loss_G_gan_lr
            loss_G = (lambda_rec * loss_rec) + (lambda_land * loss_land) +␣
↪(lambda_gan * loss_G_gan)

            # Update G
            loss_G.backward()
            torch.nn.utils.clip_grad_norm_(generator.parameters(), 10.0)
            optimizer_G.step()

            # --- Logging ---
            if current_step_stage2 % log_freq == 0:
                 pbar_stage2.set_postfix({
                    "Loss_D": f"{loss_D.item():.4f}",
                    "Loss_G": f"{loss_G.item():.4f}",
                    "Loss_Rec": f"{loss_rec.item():.4f}",
                    "Loss_Land": f"{loss_land.item():.4f}"
                 })

            # --- Checkpointing ---
            if current_step_stage2 % (save_checkpoint_freq // 5) == 0 or␣
↪current_step_stage2 == num_finetune_steps - 1: # Save more frequently
                is_best_stage2 = loss_G.item() < best_loss_g_stage2
                if is_best_stage2:
                    best_loss_g_stage2 = loss_G.item()

                checkpoint_data_stage2 = {
                    'step': current_step_stage2,
                    'generator_state_dict': generator.state_dict(),
                    'discriminator_highres_state_dict': discriminator_highres.
↪state_dict(),
                    'discriminator_lowres_audio_state_dict':␣
↪discriminator_lowres_audio.state_dict(),
                    'optimizer_G_state_dict': optimizer_G.state_dict(),
                    'optimizer_D_highres_state_dict': optimizer_D_highres.
↪state_dict(),
                    'optimizer_D_lowres_state_dict': optimizer_D_lowres.
↪state_dict(),
                    'loss_G': loss_G.item(),
                    'best_loss_g_stage2': best_loss_g_stage2,
                }
                torch.save(checkpoint_data_stage2, stage2_checkpoint_path)
                if is_best_stage2:
                    print(f"\n  New best Stage 2 loss: {best_loss_g_stage2:.4f}␣
↪at step {current_step_stage2}")
```

```
                    torch.save(checkpoint_data_stage2,␣
 ↪stage2_best_checkpoint_path)

            pbar_stage2.update(1)
            current_step_stage2 += 1

        pbar_stage2.close()
        print("\n Stage 2 Fine-tuning Finished.")

        # Save the final generator model separately for easy loading during␣
 ↪inference
        final_generator_state = {'generator_state_dict': generator.state_dict()}
        torch.save(final_generator_state, stage2_final_model_path)
        print(f"  Final Stage 2 Generator saved to {stage2_final_model_path}")

        # Store the path for the next step
        fine_tuned_lipsync_model_path = stage2_final_model_path

    else:
        print("Skipping Stage 2 fine-tuning due to setup errors.")
        fine_tuned_lipsync_model_path = None # Indicate failure

print("\nLipsync Fine-tuning (Stage 2 - Single-speaker) section finished.")
```

12 11. Lipsync Inference

[ ]:
```
# ================================================================================
# 11. LIPSYNC INFERENCE
# ================================================================================
print("="*60)
print(f"Starting Lipsync Inference for Speaker: {SPEAKER_ID}")
print("="*60)

# --- Prerequisites ---
try:
    import librosa
except ImportError:
    print(" WARNING: librosa not found. Audio processing for inference will␣
 ↪fail.")
    print("          Install with: pip install librosa")
    librosa = None

# --- Configuration ---
inference_batch_size = 4 # Process N sequences concurrently if memory allows
inference_sequence_length = 9 # Must match training sequence length
```

```
# Overlap frames between sequences for smoother transitions (e.g., T=9,␣
 ↪overlap=3 -> step=6)
frame_overlap = 3
frame_step = inference_sequence_length - frame_overlap
num_reference_frames_inference = 10 # N=10

# --- Check Prerequisites ---
if 'fine_tuned_lipsync_model_path' not in locals() or␣
 ↪fine_tuned_lipsync_model_path is None:
    print(" ERROR: Fine-tuned Lipsync model path not found or fine-tuning␣
 ↪failed.")
    print("Please run Cell 10 successfully first.")
    inference_possible = False
elif 'target_processed_data_path' not in locals() or target_processed_data_path␣
 ↪is None:
    print(f" ERROR: Processed data path for the target video not found.")
    print("Please run Data Preparation (Cell 3) successfully first.")
    inference_possible = False
elif 'manifest_path' not in locals() or not os.path.exists(manifest_path):
     print(f" ERROR: TTS Manifest file not found at {manifest_path}.")
     print("Please run Cell 7 successfully first.")
     inference_possible = False
elif 'generator' not in locals() or generator is None: # Check if generator␣
 ↪model exists
     print(f" ERROR: Generator model not instantiated.")
     print("Please run Cell 8 successfully first.")
     inference_possible = False
elif librosa is None:
    print(f" ERROR: librosa library not installed.")
    inference_possible = False
else:
    inference_possible = True

if inference_possible:
    # --- 1. Load Fine-tuned Generator Model ---
    print(f"\n--- Loading Fine-tuned Lipsync Generator:␣
 ↪{fine_tuned_lipsync_model_path} ---")
    try:
        checkpoint = torch.load(fine_tuned_lipsync_model_path,␣
 ↪map_location=DEVICE)
        generator.load_state_dict(checkpoint['generator_state_dict'])
        generator.eval() # Set model to evaluation mode
        print(" Generator weights loaded and set to eval mode.")
        model_loaded = True
    except Exception as e:
        print(f" ERROR: Failed to load fine-tuned generator: {e}")
```

```
        model_loaded = False

    # --- 2. Load Processed Data & TTS Manifest ---
    if model_loaded:
        print("\n--- Loading Processed Video Data and TTS Manifest ---")
        try:
            processed_data = load_processed_data(target_processed_data_path)
            if not processed_data: raise ValueError("load_processed_data␣
↪returned None")

            with open(manifest_path, 'r', encoding='utf-8') as f:
                tts_manifest = json.load(f)

            num_frames_total = processed_data['num_frames']
            print(f" Loaded processed data for {num_frames_total} frames.")
            print(f" Loaded TTS manifest containing synthesized audio paths.")
            data_loaded = True
        except Exception as e:
            print(f" ERROR: Failed to load data: {e}")
            data_loaded = False

    # --- 3. Prepare Inference Data Generator ---
    if data_loaded:
        print("\n--- Preparing Inference Data ---")

        # Function to load images (cached or on-demand)
        def load_frame_image(path):
            img = cv2.imread(path)
            if img is None: return None
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            img = img.astype(np.float32) / 255.0 # Normalize 0-1
            return torch.tensor(img).permute(2, 0, 1) # HWC -> CHW

        # Function to compute mel spectrogram from audio file
        def compute_mel_spectrogram(audio_path, sample_rate=SAMPLE_RATE,␣
↪n_mels=64, n_fft=1024, hop_length=256, win_length=1024):
            try:
                wav, sr = librosa.load(audio_path, sr=sample_rate)
                if sr != sample_rate:
                    wav = librosa.resample(wav, orig_sr=sr,␣
↪target_sr=sample_rate)

                mel = librosa.feature.melspectrogram(
                    y=wav, sr=sample_rate, n_fft=n_fft, hop_length=hop_length,
                    win_length=win_length, n_mels=n_mels, fmin=0.0, fmax=8000.0
                )
                mel_db = librosa.power_to_db(mel, ref=np.max)
```

```
                # Normalize (example: scale to [0, 1] or mean/std normalization)
                mel_db_norm = (mel_db - mel_db.min()) / (mel_db.max() - mel_db.
↪min() + 1e-6)

                return torch.tensor(mel_db_norm, dtype=torch.float32) #␣
↪[n_mels, T_audio]
            except Exception as e:
                print(f"\nWarning: Failed to process audio {audio_path}: {e}")
                return None

        # Pre-load/Map Synthesized Audio Paths from Manifest
        # Map frame index to the corresponding audio segment and timestamp␣
↪offset
        # This requires careful alignment based on segment timings
        # Simpler approach: Load full audio, compute mels once, then slice
        # OR: Compute mels per segment, then find corresponding mel chunk per␣
↪frame
        print("Loading and processing synthesized audio segments...")
        audio_segments_data = {} # segment_id -> (audio_path, start_time,␣
↪end_time)
        full_audio_mels = []
        last_audio_end_time = 0.0
        # Assume manifest segments are ordered by time
        for segment in tqdm(tts_manifest['segments'], desc="Processing Audio␣
↪Segments"):
            audio_path = segment.get('translated_audio_path')
            if audio_path and os.path.exists(audio_path):
                mel = compute_mel_spectrogram(audio_path)
                if mel is not None:
                    audio_frames_per_video_frame = (sample_rate / hop_length) /
↪ processed_data['fps']

                    num_expected_video_frames = (segment['end'] -␣
↪segment['start']) * processed_data['fps']
                    num_audio_frames = mel.shape[1]

                    target_audio_frames = int(num_expected_video_frames *␣
↪audio_frames_per_video_frame)

                    if num_audio_frames > target_audio_frames:
                        mel = mel[:, :target_audio_frames]
                    elif num_audio_frames < target_audio_frames:

                        padding = target_audio_frames - num_audio_frames
                        mel = F.pad(mel, (0, padding), mode='replicate')
```

```
                    full_audio_mels.append(mel)
                    last_audio_end_time = segment['end'] # Keep track for␣
↪final padding


        # Concatenate all mels
        if full_audio_mels:
            full_audio_mels_tensor = torch.cat(full_audio_mels, dim=1) #␣
↪[n_mels, T_full_audio]
            print(f" Concatenated audio mels generated. Shape:␣
↪{full_audio_mels_tensor.shape}")
        else:
            print(" ERROR: Failed to process any synthesized audio segments.")
            full_audio_mels_tensor = None
            data_loaded = False # Cannot proceed without audio

    # --- 4. Run Inference Loop ---
    if data_loaded and model_loaded and full_audio_mels_tensor is not None:
        print("\n--- Running Lipsync Inference ---")
        predicted_mouths_dir = OUTPUT_DIR / f"{TARGET_VIDEO_PATH.
↪stem}_predicted_mouths"
        predicted_mouths_dir.mkdir(parents=True, exist_ok=True)
        print(f"Predicted mouths will be saved in: {predicted_mouths_dir}")

        # List paths for masked inputs and references
        masked_input_paths = sorted(glob.glob(os.path.
↪join(processed_data['masked_inputs_dir'], "*.png")))
        reference_mask_paths = sorted(glob.glob(os.path.
↪join(processed_data['reference_masks_dir'], "*.png")))

        # Select N random reference frames (load them once)
        # Ensure reference frames exist and are valid
        valid_ref_indices = [i for i, p in enumerate(reference_mask_paths) if␣
↪processed_data['landmarks_smoothed'][i] is not None]
        if len(valid_ref_indices) < num_reference_frames_inference:
             print(f"Warning: Only {len(valid_ref_indices)} valid frames␣
↪available for references. Sampling with replacement.")
             ref_indices = random.choices(valid_ref_indices,␣
↪k=num_reference_frames_inference)
        else:
             ref_indices = random.sample(valid_ref_indices,␣
↪num_reference_frames_inference)
```

```
        reference_frames_tensor = torch.
↪stack([load_frame_image(reference_mask_paths[i]) for i in ref_indices]).
↪to(DEVICE) # [N, C, H, W]
        reference_frames_batch = reference_frames_tensor.unsqueeze(0).
↪repeat(inference_batch_size, 1, 1, 1, 1) # [B, N, C, H, W]

        # Array to store results (or save directly)
        all_predicted_mouths = [None] * num_frames_total

        generator.eval()
        with torch.no_grad():
            # Process in overlapping sequences
            for start_idx in tqdm(range(0, num_frames_total -␣
↪inference_sequence_length + 1, frame_step), desc="Lipsync Inference"):
                end_idx = start_idx + inference_sequence_length

                # --- Prepare Batch ---
                batch_masked_frames = []
                batch_audio_mels = []

                indices_in_batch = list(range(start_idx, end_idx))

                # Load masked frames for the sequence
                seq_masked_frames = [load_frame_image(masked_input_paths[i])␣
↪for i in indices_in_batch]
                # Check for None values if loading failed
                if any(f is None for f in seq_masked_frames):
                    print(f"Warning: Skipping sequence starting at {start_idx}␣
↪due to missing input frames.")
                    continue
                seq_masked_frames_tensor = torch.stack(seq_masked_frames) # [T,␣
↪C, H, W]

                # Slice corresponding audio mels
                # Calculate audio indices based on video indices
                audio_frames_per_video_frame = (SAMPLE_RATE / hop_length) /␣
↪processed_data['fps']
                audio_len_per_step = 24 # From model config/paper

                seq_audio_mels = []
                for i in indices_in_batch:
                    center_audio_frame = int(i * audio_frames_per_video_frame)
                    audio_start = max(0, center_audio_frame -␣
↪(audio_len_per_step // 2))
                    audio_end = audio_start + audio_len_per_step
```

```
                    # Ensure slice is within bounds
                    if audio_end > full_audio_mels_tensor.shape[1]:
                         # Pad if near the end
                         available_len = full_audio_mels_tensor.shape[1] -␣
↪audio_start
                         mel_slice = full_audio_mels_tensor[:, audio_start:]
                         padding = audio_len_per_step - available_len
                         mel_chunk = F.pad(mel_slice, (0, padding),␣
↪mode='replicate')
                    else:
                         mel_chunk = full_audio_mels_tensor[:, audio_start:
↪audio_end]

                    seq_audio_mels.append(mel_chunk)

               seq_audio_mels_tensor = torch.stack(seq_audio_mels) # [T,␣
↪n_mels, audio_len_per_step]

               # Add batch dimension
               batch_masked_frames_tensor = seq_masked_frames_tensor.
↪unsqueeze(0).to(DEVICE) # [1, T, C, H, W]
               batch_audio_mels_tensor = seq_audio_mels_tensor.unsqueeze(0).
↪to(DEVICE)      # [1, T, n_mels, audio_len_per_step]
               # Reference frames are already batched [B, N, C, H, W], take␣
↪first element for B=1
               batch_ref_frames_tensor = reference_frames_batch[0:1] # [1, N,␣
↪C, H, W]

               # --- Run Generator ---
               pred_images_seq, _ = generator(
                   batch_masked_frames_tensor,
                   batch_audio_mels_tensor,
                   batch_ref_frames_tensor
               )
               pred_images_seq = pred_images_seq.squeeze(0).cpu() # [T, C, H,␣
↪W]

               # --- Store/Save Results (Handle Overlap) ---
               # Average predictions in overlapping regions or just take␣
↪center part
               # Simple approach: Store only the non-overlapping part
               actual_start = frame_overlap // 2 if start_idx > 0 else 0
               actual_end = inference_sequence_length - (frame_overlap -␣
↪actual_start) if end_idx < num_frames_total else inference_sequence_length

               for i in range(actual_start, actual_end):
```

```
                    frame_idx_global = start_idx + i
                    if all_predicted_mouths[frame_idx_global] is None: # Store␣
↪if not already filled by previous overlap
                        # Convert tensor [C, H, W] back to HWC uint8 BGR for␣
↪saving
                        pred_img_np = pred_images_seq[i].permute(1, 2, 0).
↪numpy() # HWC, RGB
                        pred_img_np = (pred_img_np * 255).astype(np.uint8)
                        pred_img_bgr = cv2.cvtColor(pred_img_np, cv2.
↪COLOR_RGB2BGR)

                        # Save the predicted mouth image
                        output_img_path = predicted_mouths_dir /␣
↪f"pred_frame_{frame_idx_global:06d}.png"
                        cv2.imwrite(str(output_img_path), pred_img_bgr)
                        all_predicted_mouths[frame_idx_global] =␣
↪str(output_img_path) # Store path

        # Check if all frames were processed
        missing_frames = sum(1 for p in all_predicted_mouths if p is None)
        if missing_frames > 0:
            print(f" Warning: {missing_frames} predicted mouth frames seem to␣
↪be missing. Check overlap logic or errors.")

        print(f"\n Inference complete. Predicted mouth images saved in:␣
↪{predicted_mouths_dir}")
        predicted_mouth_paths = all_predicted_mouths # List of paths to␣
↪predicted mouths

else:
    print("Skipping Lipsync Inference due to previous errors.")
    predicted_mouth_paths = None # Indicate failure

print("\nLipsync Inference section finished.")
```

13 12. Video Rendering

[ ]:
```
# ==============================================================================
# 12. VIDEO RENDERING
# ==============================================================================
print("="*60)
print(f"Starting Video Rendering for: {TARGET_VIDEO_PATH.name}")
print("="*60)
```

```
if 'predicted_mouth_paths' not in locals() or predicted_mouth_paths is None:
    print(" ERROR: Predicted mouth paths not found.")
    print("Please run Lipsync Inference (Cell 11) successfully first.")
    rendering_possible = False
elif 'processed_data' not in locals() or processed_data is None:
    print(" ERROR: Processed data (landmarks, transforms) not loaded.")
    print("Please ensure Cell 11 loaded 'processed_data' successfully.")
    rendering_possible = False
elif not TARGET_VIDEO_PATH.exists():
    print(f" ERROR: Original video file not found at: {TARGET_VIDEO_PATH}")
    rendering_possible = False
else:
    rendering_possible = True

if rendering_possible:
    # --- Configuration ---
    rendered_frames_dir = OUTPUT_DIR / f"{TARGET_VIDEO_PATH.
 ↪stem}_rendered_frames"
    rendered_frames_dir.mkdir(parents=True, exist_ok=True)
    print(f"Rendered frames will be saved in: {rendered_frames_dir}")

    # Gaussian blur kernel size for mask feathering (must be odd)
    blur_kernel_size = 21 #
    # --- Define Rendering Functions ---

    def get_inverse_transform(frame_index: int) -> Optional[np.ndarray]:
        """Retrieves the inverse transform matrix for a given frame index."""
        if frame_index < len(processed_data['inverse_transforms']):
            return processed_data['inverse_transforms'][frame_index]
        else:
            print(f"Warning: Inverse transform for frame {frame_index} not␣
 ↪found.")
            return None

    def generate_blend_mask(landmarks_frame: np.ndarray, img_size: Tuple[int,␣
 ↪int]) -> Optional[np.ndarray]:
        """
        Generates a blurred polygonal mask based on facial landmarks.
        Follows the paper's description (convex hull, chin adjustment, blur).
        """
        if landmarks_frame is None or landmarks_frame.size == 0:
            return None #

        left_ear_idx = 34 # Example
        right_ear_idx = 264 # Example
        nose_halfway_idx = 6 # Example (bridge)
        nose_tip_idx = 1 # Example
```

```
        chin_left_idx = 132 # Example
        chin_right_idx = 361 # Example
        chin_center_idx = 152 # Example

        try:
            points_indices = [
                left_ear_idx, right_ear_idx,
                nose_halfway_idx, nose_tip_idx,
                chin_left_idx, chin_right_idx, chin_center_idx
            ]

            # Select the points (x, y coordinates)
            points = landmarks_frame[points_indices, :2].astype(np.int32)
            mouth_top_idx = 13 # Example upper lip
            mouth_bottom_idx = 14 # Example lower lip
            chin_shift = (landmarks_frame[mouth_bottom_idx, 1] -␣
↪landmarks_frame[mouth_top_idx, 1]) * 1.5 # Shift down by 1.5x mouth height

            # Apply shift to chin indices in the 'points' array
            # Assuming chin_left, chin_right, chin_center are the last 3 points
            points[-3:, 1] += int(chin_shift)

            # Calculate convex hull
            hull = cv2.convexHull(points)

            # Create binary mask
            mask = np.zeros(img_size[:2], dtype=np.float32) # Use float32 for␣
↪blurring
            cv2.fillConvexPoly(mask, hull, 1.0)

            # Apply Gaussian blur for feathering
            # Kernel size must be odd
            k_size = blur_kernel_size
            if k_size % 2 == 0: k_size += 1

            blurred_mask = cv2.GaussianBlur(mask, (k_size, k_size), 0)

            # Ensure mask values are clamped between 0 and 1
            blurred_mask = np.clip(blurred_mask, 0.0, 1.0)

            # Add channel dimension for blending
            return blurred_mask[:, :, np.newaxis] # Shape (H, W, 1)

        except IndexError:
             print("Warning: Landmark indices out of bounds. Cannot generate␣
↪mask.")
             return None
```

```
        except Exception as e:
            print(f"Warning: Error generating mask: {e}")
            return None


    def blend_frame(original_frame: np.ndarray,
                    predicted_mouth_crop: np.ndarray,
                    inverse_transform: np.ndarray,
                    blend_mask: np.ndarray) -> np.ndarray:
        """
        Blends the predicted mouth back into the original frame.
        """
        H, W = original_frame.shape[:2]

        # Warp predicted mouth crop back to original frame coordinates
        warped_mouth = cv2.warpAffine(
            predicted_mouth_crop,
            inverse_transform,
            (W, H),
            flags=cv2.INTER_CUBIC # Use high-quality interpolation
        )

        # Warp the blend mask similarly
        warped_mask = cv2.warpAffine(
            blend_mask,
            inverse_transform,
            (W, H),
            flags=cv2.INTER_LINEAR # Linear is fine for masks
        )
        # Ensure mask has 3 channels if it lost one during warp, or frame is␣
↪grayscale
        if warped_mask.ndim == 2:
            warped_mask = warped_mask[:, :, np.newaxis]
        if warped_mask.shape[2] == 1 and original_frame.shape[2] == 3:
             warped_mask = cv2.cvtColor(warped_mask * 255, cv2.COLOR_GRAY2BGR) /
↪ 255.0
             # Re-clip after potential conversion artifacts
             warped_mask = np.clip(warped_mask, 0.0, 1.0)


        # Ensure original frame is float32 for blending
        original_frame_float = original_frame.astype(np.float32) / 255.0
        warped_mouth_float = warped_mouth.astype(np.float32) / 255.0

        # Alpha blending: Combine the warped mouth and original frame
        # Formula: output = mask * foreground + (1 - mask) * background
```

```
        blended_float = warped_mask * warped_mouth_float + (1.0 - warped_mask)␣
↪* original_frame_float

        # Convert back to uint8
        blended_uint8 = np.clip(blended_float * 255.0, 0, 255).astype(np.uint8)

        return blended_uint8

    # --- Iteration Loop ---
    print("\n--- Iterating through frames for rendering ---")
    num_frames_to_render = processed_data['num_frames']
    cap = cv2.VideoCapture(str(TARGET_VIDEO_PATH)) # Reopen original video

    for i in tqdm(range(num_frames_to_render), desc="Rendering Frames"):
        success, original_frame = cap.read()
        if not success:
            print(f"Warning: Failed to read original frame {i}. Stopping.")
            break

        # --- Load Components for this frame ---
        pred_mouth_path = predicted_mouth_paths[i]
        landmarks = processed_data['landmarks_smoothed'][i]
        inv_transform = get_inverse_transform(i)

        # Check if all components are valid
        if pred_mouth_path is None or not os.path.exists(pred_mouth_path):
            print(f"Warning: Predicted mouth for frame {i} not found. Saving␣
↪original frame.")
            output_frame = original_frame
        elif landmarks is None:
            print(f"Warning: Landmarks for frame {i} not found. Cannot generate␣
↪mask. Saving original frame.")
            output_frame = original_frame
        elif inv_transform is None:
            print(f"Warning: Inverse transform for frame {i} not found. Saving␣
↪original frame.")
            output_frame = original_frame
        else:
            predicted_mouth_img = cv2.imread(pred_mouth_path)
            if predicted_mouth_img is None:
                print(f"Warning: Failed to load predicted mouth␣
↪{pred_mouth_path}. Saving original frame.")
                output_frame = original_frame
            else:
                # --- Generate Mask ---
                blend_mask = generate_blend_mask(landmarks, original_frame.
↪shape)
```

```
                if blend_mask is None:
                    print(f"Warning: Failed to generate blend mask for frame␣
 ↪{i}. Saving original frame.")
                    output_frame = original_frame
                else:
                    # --- Blend Frame ---
                    output_frame = blend_frame(
                        original_frame,
                        predicted_mouth_img,
                        inv_transform,
                        blend_mask
                    )

            # --- Save Rendered Frame ---
            output_frame_path = rendered_frames_dir / f"rendered_frame_{i:06d}.png"
            cv2.imwrite(str(output_frame_path), output_frame)

    cap.release()
    print("\n Frame rendering complete.")
    rendered_frame_paths_pattern = str(rendered_frames_dir /␣
 ↪"rendered_frame_%06d.png") # Pattern for ffmpeg

else:
    print("Skipping Video Rendering due to previous errors.")
    rendered_frame_paths_pattern = None # Indicate failure

print("\nVideo Rendering section finished.")
```

14 13. Final Video Assembly

[ ]:
```
# ==============================================================================
# 13. FINAL VIDEO ASSEMBLY
# ==============================================================================
print("="*60)
print(f"Starting Final Video Assembly for: {TARGET_VIDEO_PATH.name}")
print("="*60)

try:
    import soundfile as sf
except ImportError:
    print(" WARNING: soundfile library not found. Audio concatenation will fail.
 ↪")
    print("         Install with: pip install soundfile")
    sf = None
```

```
if 'synthesized_segments_info' not in locals() or synthesized_segments_info is␣
 ↪None:
    print(" ERROR: Synthesized audio segment information not found.")
    print("Please run TTS Synthesis (Cell 7) successfully first.")
    assembly_possible = False
elif 'rendered_frame_paths_pattern' not in locals() or␣
 ↪rendered_frame_paths_pattern is None:
    print(" ERROR: Rendered frame path pattern not found.")
    print("Please run Video Rendering successfully first.")
    assembly_possible = False
elif sf is None:
    print(" ERROR: soundfile library is required for audio concatenation.")
    assembly_possible = False
else:
    assembly_possible = True

if assembly_possible:
    # --- 1. Concatenate Synthesized Audio Segments ---
    print("\n--- 1. Concatenating Audio Segments ---")
    concatenated_audio_path = OUTPUT_DIR / f"{TARGET_VIDEO_PATH.
 ↪stem}_final_audio_{TARGET_LANG}.wav"

    audio_data_list = []
    expected_sample_rate = TTSComponent.VOCODER_SAMPLE_RATE # Get SR from TTS␣
 ↪Component
    total_samples = 0

    # Sort segments by start time just in case
    sorted_segments = sorted(synthesized_segments_info, key=lambda x: x.
 ↪get('start', 0))

    for i, segment in enumerate(tqdm(sorted_segments, desc="Reading Audio␣
 ↪Segments")):
        audio_path = segment.get('audio_path')
        if audio_path and os.path.exists(audio_path):
            try:
                data, samplerate = sf.read(audio_path, dtype='float32')
                if samplerate != expected_sample_rate:
                    #
                    print(f"Warning: Segment {i} has sample rate {samplerate},␣
 ↪expected {expected_sample_rate}. Skipping resampling for now.")


                audio_data_list.append(data)
                total_samples += len(data)
            except Exception as e:
```

```
                print(f"\nWarning: Failed to read audio segment {audio_path}:␣
↪{e}")
        else:

            duration = segment.get('duration', 0)
            if duration > 0:
                silence = np.zeros(int(duration * expected_sample_rate),␣
↪dtype=np.float32)
                audio_data_list.append(silence)
                total_samples += len(silence)
                print(f"Warning: Audio for segment {segment.get('id', i)}␣
↪missing. Adding {duration:.2f}s silence.")

    if audio_data_list:
        try:
            final_audio_data = np.concatenate(audio_data_list)
            sf.write(str(concatenated_audio_path), final_audio_data,␣
↪expected_sample_rate)
            print(f"\n Concatenated audio saved to: {concatenated_audio_path}")
            print(f"  Total duration: {total_samples / expected_sample_rate:.
↪2f} seconds")
            audio_ready = True
        except Exception as e:
            print(f" ERROR: Failed to concatenate or write final audio: {e}")
            audio_ready = False
    else:
        print(" ERROR: No valid audio segments found to concatenate.")
        audio_ready = False

    # --- 2. Combine Frames and Audio using FFmpeg ---
    if audio_ready:
        print("\n--- 2. Combining Rendered Frames and Audio using FFmpeg ---")
        final_video_output_path = OUTPUT_DIR / f"{TARGET_VIDEO_PATH.
↪stem}_dubbed_{TARGET_LANG}.mp4"

        # Get frame rate from processed data
        fps = processed_data.get('fps', VIDEO_FPS) # Use default if not found

        # Construct FFmpeg command
        cmd = [
            'ffmpeg',
            '-y',  # Overwrite output file if it exists
            '-r', str(fps), # Input frame rate
            '-i', rendered_frame_paths_pattern, # Input image sequence pattern
            '-i', str(concatenated_audio_path), # Input audio file
            '-c:v', 'libx264', # Video codec
```

```
            '-pix_fmt', 'yuv420p', # Pixel format for compatibility
            '-c:a', 'aac', # Audio codec
            '-b:a', '192k', # Audio bitrate
            '-shortest', # Finish encoding when the shortest input stream ends
            str(final_video_output_path)
        ]

        print("\nExecuting FFmpeg command:")
        print(f"  {' '.join(cmd)}")

        try:
            # Run FFmpeg
            process = subprocess.run(cmd, check=True, stdout=subprocess.PIPE,␣
↪stderr=subprocess.PIPE, universal_newlines=True)
            print("\n FFmpeg execution successful.")
            print(f" Final dubbed video saved to: {final_video_output_path}")



        except subprocess.CalledProcessError as e:
            print(f" ERROR: FFmpeg command failed with exit code {e.returncode}.
↪")
            print("--- FFmpeg stderr ---")
            print(e.stderr)
            print("\nPlease check the FFmpeg command and ensure FFmpeg is␣
↪installed correctly.")
        except FileNotFoundError:
            print(" ERROR: 'ffmpeg' command not found.")

        except Exception as e:
            print(f" ERROR: An unexpected error occurred during FFmpeg␣
↪execution: {e}")
    else:
        print("Skipping final video assembly because audio concatenation failed.
↪")

else:
    print("Skipping Final Video Assembly due to previous errors.")

print("\nFinal Video Assembly section finished.")
```

15 14. Evaluation

[ ]:
```
# ==============================================================================
# 14. EVALUATION
# ==============================================================================
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim
from IPython.display import display, Video, HTML

print("="*60)
print(f"Starting Evaluation for: {TARGET_VIDEO_PATH.name}")
print("="*60)

# --- Prerequisites ---
if 'final_video_output_path' not in locals() or not os.path.
 ↪exists(final_video_output_path):
    print(" ERROR: Final dubbed video path not found.")
    print("Please run Final Video Assembly (Cell 13) successfully first.")
    evaluation_possible = False
elif 'processed_data' not in locals() or processed_data is None:
    print(" ERROR: Processed data (original crops, landmarks) not loaded.")
    print("Please ensure Cell 11 loaded 'processed_data' successfully.")
    evaluation_possible = False
elif 'rendered_frames_dir' not in locals() or not rendered_frames_dir.exists():
    print(" ERROR: Rendered frames directory not found.")
    print("Please run Video Rendering (Cell 12) successfully first.")
    evaluation_possible = False
else:
    evaluation_possible = True

if evaluation_possible:
    # --- 1. Display Final Video ---
    print("\n--- 1. Displaying Final Dubbed Video ---")
    try:
        display(Video(str(final_video_output_path), embed=True, width=640))
    except Exception as e:
        print(f" ERROR: Failed to display video: {e}")
        # Provide a link as fallback
        print(f"  > view the video manually at: {final_video_output_path}")

    # --- 2. Calculate Objective Metrics (PSNR, SSIM on Mouth Region) ---
    print("\n--- 2. Calculating Objective Metrics (PSNR, SSIM) on Mouth Region␣
 ↪---")

    # Load original cropped faces (ground truth)
    original_cropped_dir = os.path.join(processed_data['cropped_faces_dir'])
```

```
    # Load rendered frames (containing the blended mouth)
    rendered_frames_paths = sorted(glob.glob(os.path.
↪join(str(rendered_frames_dir), "*.png")))

    psnr_values = []
    ssim_values = []
    frames_compared = 0

    # Define mouth region based on the mask used during preprocessing
    # (x1, y1, x2, y2) -> (left, top, right, bottom) in normalized coords
    norm_mask_rect = processed_data.get('mask_rect_norm', (0.08, 0.28, 0.92, 0.
↪95))
    output_size = processed_data.get('output_size', 256)
    mouth_x1 = int(norm_mask_rect[0] * output_size)
    mouth_y1 = int(norm_mask_rect[1] * output_size)
    mouth_x2 = int(norm_mask_rect[2] * output_size)
    mouth_y2 = int(norm_mask_rect[3] * output_size)

    num_frames_to_eval = min(processed_data['num_frames'],␣
↪len(rendered_frames_paths))

    for i in tqdm(range(num_frames_to_eval), desc="Calculating Metrics"):
        # Load original cropped face
        original_crop_path = os.path.join(original_cropped_dir, f"frame_{i:06d}.
↪png")
        if not os.path.exists(original_crop_path): continue
        original_crop = cv2.imread(original_crop_path)
        if original_crop is None: continue

        # Load rendered frame
        rendered_frame = cv2.imread(rendered_frames_paths[i])
        if rendered_frame is None: continue



        pred_mouth_path = predicted_mouth_paths[i] # From Cell 11
        if pred_mouth_path is None or not os.path.exists(pred_mouth_path):␣
↪continue
        predicted_mouth_crop = cv2.imread(pred_mouth_path)
        if predicted_mouth_crop is None: continue

        # Extract mouth region from original cropped face
        original_mouth = original_crop[mouth_y1:mouth_y2, mouth_x1:mouth_x2]
        # Extract mouth region from predicted mouth crop
        predicted_mouth = predicted_mouth_crop[mouth_y1:mouth_y2, mouth_x1:
↪mouth_x2]
```

```
        # Calculate metrics if regions are valid
        if original_mouth.size > 0 and predicted_mouth.size == original_mouth.
↪size:
            try:
                # PSNR (higher is better)
                psnr_val = psnr(original_mouth, predicted_mouth, data_range=255)
                psnr_values.append(psnr_val)


                ssim_val = ssim(original_mouth, predicted_mouth,␣
↪data_range=255, channel_axis=-1, win_size=7) # Use smaller window for small␣
↪regions
                ssim_values.append(ssim_val)
                frames_compared += 1
            except Exception as e:

                pass #

    if frames_compared > 0:
        avg_psnr = np.mean(psnr_values)
        avg_ssim = np.mean(ssim_values)
        print(f"\n Objective Metrics Calculated ({frames_compared}/
↪{num_frames_to_eval} frames):")
        print(f"  Average PSNR: {avg_psnr:.2f} dB")
        print(f"  Average SSIM: {avg_ssim:.4f}")

        # Compare with paper's results (from our expectation table)
        print("\nComparison with Paper's Reported Best:")
        print(f"  PSNR: Baseline={avg_psnr:.2f} vs Paper=31.38 dB")
        print(f"  SSIM: Baseline={avg_ssim:.4f} vs Paper=0.970")
        if avg_psnr < 30 or avg_ssim < 0.96:
             print("  (As expected, baseline performance is lower due to␣
↪smaller dataset)")
    else:
        print("\nCould not calculate objective metrics. Check input data and␣
↪mouth region extraction.")

    # --- 3. FID (Fréchet Inception Distance) ---
    print("\n--- 3. FID (Fréchet Inception Distance) ---")
    print("FID requires comparing distributions of features from many generated␣
↪images")
    print("vs. many real images using a pre-trained Inception model.")
    print("Calculating it for a single video is less meaningful and complex to␣
↪set up here.")
```

```
    print("Consider calculating FID across a test set of generated videos as a␣
 ↪further step.")

    # --- 4. Subjective Viewing ---


else:
    print("Skipping Evaluation due to previous errors.")

print("\nEvaluation section finished.")
```